

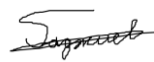
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Московский институт электроники и математики им. А.Н. Тихонова

Гаджиев Эмиль Гадирович, группа БИБ224

**АЛГОРИТМЫ ВСТРАИВАНИЯ ЦИФРОВЫХ
ВОДЯНЫХ ЗНАКОВ ДЛЯ РАСПОЗНАВАНИЯ СГЕНЕРИРОВАННЫХ
ОБЪЕКТОВ НА ИЗОБРАЖЕНИЯХ**

Выпускная квалификационная работа
по направлению 10.03.01 «Информационная безопасность»
студента образовательной программы бакалавриата
«Информационная безопасность»

Студент  Э.Г. Гаджиев

Руководитель
Приглашенный преподаватель,
кафедра информационной безопасности
киберфизических систем департамента
кибербезопасности МИЭМ НИУ ВШЭ



К.М. Джанашиа

Москва 2026 г.

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Московский институт электроники и математики им. А.Н. Тихонова

**ЗАДАНИЕ
на выполнение выпускной квалификационной работы**

студенту группы БИБ224 Гаджиеву Эмилю Гадировичу

1. Тема работы

Алгоритмы встраивания цифровых водяных знаков для распознавания сгенерированных объектов на изображениях.

2. Требования к работе

2.1. Цель работы

Повышение точности детектирования сгенерированных объектов на изображениях за счет встраивания полухрупких цифровых водяных знаков с устойчивостью к санкционированным искажениям.

2.2. Требования к результатам работы

– Должна быть разработана или улучшена полухрупкая схема встраивания цифровых водяных знаков с качеством локализации фальсификаций не менее 0,8 по метрике F1 при наличии искажающих воздействий.

– Должен быть создан программный модуль, реализующий предлагаемую схему и позволяющий проводить вычислительные эксперименты по оценке эффективности разработанной схемы и ее сравнению с аналогами.

– Эффективность предложенной схемы по критериям точности обнаружения фальсификаций, устойчивости к искажающим воздействиям и незаметности цифрового водяного знака должна быть оценена с помощью вычислительных экспериментов.

2.3. Требования к документации

Текст ВКР должен быть оформлен в соответствии с ГОСТ.

3. Содержание работы

3.1. Обзор научно–технической литературы по теме работы.

3.2. Рассмотрение различных подходов к организации встраивания полухрупких цифровых водяных знаков.

3.3. Разработка или совершенствование схемы полухрупкого встраивания цифровых водяных знаков. Подготовка программной реализации.

3.4. Проведение вычислительных экспериментов для оценки схемы, в том числе для ее сравнения с аналогами.

3.5. Подготовка пояснительной записки к ВКР.

4. Сроки выполнения этапов работы

Проект ВКР представляется студентом в срок до «15» февраля 2026 г.

Первый вариант ВКР представляется студентом в срок до «15» апреля 2026 г.

Итоговый вариант ВКР представляется студентом
руководителю до загрузки работы в систему «30» апреля 2026 г.
«Антиплагиат» в срок до

Задание выдано

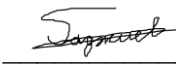
«25» декабря 2025 г.


подпись руководителя

К.М. Джанашиа

Задание принято к
исполнению

«25» декабря 2025 г.


подпись студента

Э.Г. Гаджиев

АННОТАЦИЯ

Распространение генеративных моделей и инструментов редактирования изображений повышает риск создания и распространения измененного визуального контента, для которого сложно подтвердить происхождение и выявить область вмешательства. Одним из способов решения этой проблемы является встраивание цифровых водяных знаков, позволяющих связать изображение со скрытым защитным сигналом и выполнить его последующую проверку. В работе рассматриваются алгоритмы встраивания цифровых водяных знаков для распознавания сгенерированных объектов на изображениях. Основное внимание уделено полухрупким цифровым водяным знакам, которые должны сохраняться при допустимой обработке изображения и реагировать на содержательные изменения. В качестве основы выбран метод OmniGuard, поддерживающий локализацию вмешательств и извлечение авторской битовой метки.

В ходе работы исходная реализация OmniGuard была адаптирована для воспроизводимого локального запуска, после чего был разработан усовершенствованный вариант метода. В него добавлены структурированная служебная битовая метка, гибридная карта вмешательства и экспериментальный контур для пакетной проверки. Сравнение с исходным OmniGuard показало повышение качества локализации области вмешательства по метрике F1, что соответствует цели повышения точности детектирования сгенерированных объектов на изображениях.

Работа содержит 71 страницу, 14 рисунков, 13 таблиц, 31 источник и 1 приложение.

ABSTRACT

The spread of generative models and image editing tools increases the risk of creating and distributing modified visual content whose origin is difficult to confirm and whose edited areas are difficult to detect. One way to solve this problem is to embed digital watermarks, which make it possible to link an image with a hidden protective signal and verify it later.

The thesis examines digital watermarking algorithms for recognizing and verifying generated objects in images. The main focus is on semi-fragile schemes, which should remain stable under acceptable image processing and respond to meaningful content changes. OmniGuard was chosen as the base method, as it supports tamper localization and extraction of an author bit mark.

During the work, the original OmniGuard implementation was adapted for reproducible local execution. After that, an improved version of the method was developed. It includes a structured service bit mark, a hybrid tampering map, and an experimental pipeline for batch testing.

The experiments showed that the improved OmniGuard outperforms LSB and the method based on the discrete cosine transform in terms of robustness of embedded data extraction after typical image transformations.

An additional comparison with the original OmniGuard showed an improvement in the quality of tamper localization according to the F1 metric, which corresponds to the goal of improving the accuracy of detecting generated objects in images.

The thesis contains 71 pages, 14 illustrations, 13 tables, 31 references, and 1 appendix.

Содержание

Список используемых терминов и сокращений	8
Введение.....	10
1. Литературный обзор	12
1.1 Историческая справка.....	12
1.2 Современные подходы	18
1.3 Метрики оценки качества встраивания ЦВЗ.....	23
1.4 Атаки на цифровые водяные знаки	25
1.5 Выводы по литературному обзору	28
2. Базовый метод встраивания с поддержкой распознавания сгенерированных изображений	30
2.1 Выбор базового метода.....	30
2.2 OmniGuard.....	32
2.3 Актуализация существующего решения OmniGuard	35
2.4 Эффективность OmniGuard.....	38
3. Усовершенствованный метод встраивания с поддержкой распознавания сгенерированных изображений	40
3.1 Проверяемая и устойчивая служебная битовая метка	40
3.2 Двухкомпонентная структура цифрового водяного знака	42
3.3 Проблема построения информативной карты вмешательства.....	43
3.4 Проблема ограниченности исходного демонстрационного сценария	44
4. Экспериментальный стенд	47
4.1 Общая логика работы алгоритма в стенде	48
4.2 Основной пользовательский сценарий	49

4.3 Режим сравнения методов	51
4.4 Пакетная экспериментальная проверка	52
4.5 Режим сравнения с исходным OmniGuard	53
4.6 Выводы по разделу	55
5. Экспериментальная часть.....	56
5.1 Постановка эксперимента	56
5.2 Незаметность контейнерного изображения	57
5.3 Устойчивость встроенной служебной информации.....	57
5.4 BER по видам воздействий	58
5.5 Сравнение исходного и усовершенствованного OmniGuard.....	60
5.6 Выводы по эксперименту	64
Выводы.....	66
Список использованной литературы.....	68
Приложение А. Описание программной реализации и ключевые фрагменты кода	72

Список используемых терминов и сокращений

В настоящей работе применяют следующие сокращения и обозначения:

AIGC – artificial intelligence generated content, контент, сгенерированный искусственным интеллектом

BER – bit error rate, доля битовых ошибок при извлечении сообщения

Bpp – bits per pixel, число бит на пиксель

HMAC – hash-based message authentication code, код проверки целостности сообщения на основе ключа

IoU – intersection over union, отношение площади пересечения масок к площади их объединения

ДКП – discrete cosine transform, дискретное косинусное преобразование

ДВП – discrete wavelet transform, дискретное вейвлет-преобразование

ДПФ – discrete fourier transform, дискретное преобразование Фурье

EditGuard – нейросетевой метод цифрового водяного маркирования изображений для защиты авторства и локализации правок

FNR – false negative rate, доля пропусков при обнаружении водяного знака или области вмешательства

FPR – false positive rate, доля ложных срабатываний при обнаружении водяного знака или области вмешательства

F1-score – гармоническая средняя точности и полноты при оценке маски вмешательства

LSB – least significant bit, метод замены наименее значимых битов

mAP – mean average precision, средняя точность обнаружения или локализации

MSE – mean squared error, среднеквадратичная ошибка между исходным и контейнерным изображениями

NCC – normalized cross-correlation, нормализованная корреляция между исходной и извлеченной меткой

OmniGuard – гибридный нейросетевой метод цифрового водяного маркирования изображений для подтверждения авторства и локализации правок

PSNR – peak signal-to-noise ratio, пиковое отношение сигнал/шум

PVD – pixel value differencing, метод разности значений пикселей

QIM – quantization index modulation, метод модуляции индекса квантования

SSIM – structural similarity index measure, индекс структурного сходства изображений

SVD – singular value decomposition, сингулярное разложение

ЦВЗ – цифровой водяной знак

Введение

Развитие интернета в последние десятилетия позволило проще копировать, изменять, воспроизводить и распространять цифровые изображения с минимальными затратами и практически мгновенной доставкой без потери качества. С ростом развития сетевых технологий появились большие угрозы для конфиденциальности и безопасности передаваемых данных. Поэтому аутентификация контента, защита авторских прав и защита от копирования играют важную роль в борьбе с существующими и будущими угрозами, связанными с хранением цифровой информации. Одним из возможных инструментов для достижения этих целей стали цифровые водяные знаки (ЦВЗ).

Цифровые водяные знаки представляют собой особые метки, встраиваемые в цифровой объект по некоему алгоритму. Они зачастую могут содержать данные об авторе (правообладателе), информацию о самом объекте, метки времени и иные объекты.

Главными задачами в работе с цифровыми водяными знаками являются:

- защита авторских прав;
- мониторинг вещания;
- управление копированием и воспроизведением;
- фильтрация содержимого.

Для оценки эффективности встраивания информации в цифровое изображение используют три основных показателя:

- незаметность – показывает, насколько слабо встраивание влияет на визуальное качество изображения;
- емкость – отражает объем информации, который можно встроить;
- робастность – характеризует способность метки корректно обнаруживаться или извлекаться после типовых воздействий и преобразований.

Встраивание цифровых водяных знаков [1, 2], подобно стеганографии, представляет собой задачу внедрения информации в изображение визуально

незаметным образом. Эти методы обычно делят на три основных типа: хрупких, устойчивых и полухрупких [3, 10, 11, 12]. Хрупкие и полухрупкие водяные знаки чаще всего используются для проверки целостности и подлинности изображения. Хрупкий водяной знак подходит для строгой аутентификации цифрового контента: даже минимальное изменение приводит к тому, что изображение не проходит проверку. Устойчивые водяные знаки, напротив, должны сохраняться после различных преобразований изображения. Благодаря этому правообладатель может подтвердить принадлежность контента даже в тех случаях, когда изображение было повторно опубликовано, сжато или частично изменено. Полухрупкие водяные знаки сочетают в себе преимущества как стойких, так и хрупких водяных знаков и используются в основном для аутентификации цифровых изображений и выявления подделок. Для задачи распознавания сгенерированных объектов особенно важны полухрупкие схемы, позволяющие не только обнаруживать факт вмешательства, но и локализовать измененную область.

Актуальность исследования связана с тем, что генеративное редактирование позволяет создавать реалистичные изменения изображений, которые сложно обнаружить только визуально. Цель работы – повышение точности детектирования сгенерированных объектов на изображениях за счет встраивания полухрупких цифровых водяных знаков с устойчивостью к санкционированным искажениям. Для достижения цели необходимо выполнить обзор научно–технической литературы, рассмотреть подходы к организации встраивания полухрупких цифровых водяных знаков, разработать или усовершенствовать схему полухрупкого встраивания, подготовить программную реализацию и провести вычислительные эксперименты для оценки схемы и ее сравнения с аналогами.

1. Литературный обзор

В первой главе рассматриваются основные подходы к цифровому водяному маркированию изображений. Сначала описываются классические пространственные и частотные методы, затем современные нейросетевые схемы, после чего приводятся метрики оценки и основные группы атак на цифровые водяные знаки.

В настоящей работе под цифровым водяным знаком понимается метка, встраиваемая в изображение алгоритмическим способом и используемая для подтверждения происхождения, проверки целостности или локализации области вмешательства.

1.1 Историческая справка

Развитие цифровых водяных знаков связано с практическими угрозами, которые возникали по мере того, как изображения становились массовым цифровым продуктом. Для таких изображений важны защита от несанкционированного копирования, подтверждение происхождения, проверка целостности и определение области вмешательства. Поэтому в литературе рассматриваются разные классы методов: пространственные, частотные, робастные, хрупкие и полухрупкие.

Один из наиболее простых классов методов строится вокруг прямой модификации пикселей. В этом классе наиболее известны методы наименее значащего бита LSB (Least Significant Bit) [9, 10]: биты сообщения записываются в младшие разряды пикселей. С инженерной точки зрения это объяснимо: LSB-замена проста, требует минимум вычислений и обладает высокой емкостью. Однако исторически довольно быстро стало ясно, что устойчивость таких схем ограничена. JPEG-сжатие [11], пересохранение, фильтрация, изменение размера и даже умеренный шум легко нарушают младшие биты – а значит, метка либо теряется, либо становится нечитаемой.

В рамках пространственных подходов используются методы, которые сохраняют простоту LSB-встраивания, но улучшают незаметность и скрытность внедрения. Так, PM1 [12] основан на том, что при несоответствии требуемого бита и текущего младшего разряда значение изменяют минимально – увеличивают или уменьшают на единицу. Преимущество PM1 в том, что изменения выглядят как естественные малые колебания и обычно менее заметны, чем при прямой «перезаписи» младшего разряда, а также снижаются некоторые статистические признаки, по которым простые LSB-схемы могут обнаруживаться. Другой важный класс – PVD (Pixel Value Differencing) [13, 14], где встраивание опирается на разности соседних значений: в текстурных областях и на границах допускается более активное внедрение, а на гладких участках – более осторожное. За счет такой адаптации PVD обеспечивает хороший компромисс между емкостью и незаметностью, поскольку распределяет нагрузку там, где человеческое зрение хуже фиксирует небольшие изменения.

Еще одним важным методом является QIM (Quantization Index Modulation) [15, 16], который кодирует информацию через квантование: значение приводится к одному из заранее заданных уровней, и выбор группы уровней соответствует встраиваемому биту. Сильная сторона QIM – управляемость: шаг квантования задает баланс между незаметностью и устойчивостью, а извлечение обычно более стабильно при наличии округлений и умеренных искажений, чем у прямого побитового встраивания.

Отдельную группу образуют робастные методы, предназначенные прежде всего для подтверждения авторства. Их основная идея состоит в том, чтобы не размещать метку в одной небольшой области, а распределять ее по изображению. Типичный пример такого подхода – подходы «spread spectrum» (расширенный спектр) [17, 18], где псевдослучайный сигнал, задаваемый ключом, добавляется к контейнеру так, чтобы он был визуально незаметен, но одновременно с этим статистически обнаружим корреляционными методами. За счет распределения

сигнала по изображению повышается вероятность успешного обнаружения метки после сжатия, добавления шума или другой умеренной обработки: даже если часть встроенного сигнала повреждается, оставшиеся фрагменты могут сохранять достаточную корреляцию с эталоном. Но на практике оказалось, что даже робастный сигнал легко обойти обрезкой, масштабированием или поворотом: метка просто перестает совпадать с ожидаемым паттерном при извлечении.

Наряду с робастными методами отдельно развивалось направление, связанное с аутентификацией изображений. В этих задачах важно не только подтвердить происхождение цифрового объекта, но и определить, было ли в него внесено вмешательство. Именно это привело к распространению хрупких и полухрупких водяных знаков. Логика этих схем принципиально отличается от робастных: если робастный знак должен выдерживать множество обработок, то хрупкий должен разрушаться при любых изменениях, а полухрупкий – различать допустимые преобразования и содержательные правки.

Полухрупкие водяные знаки часто рассматриваются в связи с JPEG–сжатием, поскольку JPEG является одним из распространенных форматов хранения и передачи изображений. В таких схемах удобно работать в области ДКП и учитывать блочную природу JPEG, например разбиение изображения на блоки 8×8 [1, 4, 5].

ДКП (discrete cosine transform, дискретное косинусное преобразование) [19, 20] – это математический способ представить изображение или его фрагмент не через значения пикселей, а через сумму косинусных волн разной частоты и амплитуды. Методы на основе ДКП позволяют улучшить устойчивость к преобразованиям, например JPEG–сжатию, умеренному добавлению шумов и небольшим изменениям яркости, но только частично устойчивы к масштабированию, обрезке и сдвигам.

В дополнение к ДКП существуют другие преобразования, которые преобразуют изображение в формат, более подходящий для управляемого встраивания и последующей надежной проверки. Все частотные методы строятся на схожем принципе: вместо обработки исходных значений изображения, они применяют преобразование, которое разлагает информацию на «масштабы» или категории изменений. Такой подход позволяет отбирать компоненты, в которых внедрение обеспечивает наилучшее соотношение между незаметностью и устойчивостью к искажениям.

Одним из наиболее распространенных альтернативных подходов является ДВП (Discrete Wavelet Transform, дискретное вейвлет–преобразование) [19, 21]. После ДВП изображение представляется набором поддиапазонов, что позволяет более точно выбирать место внедрения: в компоненты, связанные с крупной структурой, – для большей устойчивости, либо в компоненты деталей – для повышения скрытности. Практический плюс ДВП для ЦВЗ состоит в гибкости настройки: можно регулировать глубину разложения и выбирать поддиапазоны так, чтобы метка сохранялась при сжатии и умеренной фильтрации, но при этом оставалась визуально незаметной. Дополнительно вейвлет–разложение удобнее при работе с локальными изменениями, так как оно позволяет аккуратнее контролировать влияние внедрения на разные участки и на разные масштабы изображения.

В статье [22] рассматривается метод SVD (сингулярное разложение). В рамках SVD изображение (или блок изображения) раскладывается на компоненты, при этом ключевую роль играют сингулярные значения – параметры, которые сравнительно стабильно изменяются при умеренных воздействиях. Поэтому многие схемы внедряют метку через контролируемое изменение сингулярных значений. Преимущество SVD состоит в том, что такие значения достаточно устойчивы к ряду типовых воздействий, включая умеренный шум, слабую фильтрацию и некоторые виды сжатия. При этом

визуальное качество изображения обычно остается на хорошем уровне. Для повышения устойчивости к отдельным геометрическим преобразованиям в работах [23, 24] также рассматриваются методы на основе ДПФ/БПФ (дискретного преобразования Фурье и его быстрой реализации). Преобразование Фурье позволяет представить изображение через набор синусоидальных составляющих. Такое спектральное представление может использоваться при построении схем, менее чувствительных к сдвигам, а также при разработке подходов, повышающих устойчивость к повороту и масштабированию. Практическое преимущество ДПФ-подходов заключается в том, что они потенциально позволяют получить более устойчивые к геометрическим изменениям схемы по сравнению с простыми блочными методами. При этом такие методы обычно сложнее в реализации и настройке, а также чувствительны к обработке границ и к изменениям, затрагивающим глобальную структуру изображения, поэтому их применяют там, где геометрическая устойчивость критична и оправдывает усложнение модели.

В таблице 1 приведены основные исторические методы встраивания ЦВЗ.

Таблица 1. Исторические методы встраивания ЦВЗ

Метод	Тип встраивания	Краткое описание
LSB (Least Significant Bits)	Пространственное встраивание	Метод замены наименее значимых битов (Least Significant Bits, LSB) предполагает замену одного или нескольких младших битов пикселя битами сообщения.
PM1 (Plus-Minus One)	Пространственное встраивание	Метод «плюс-минус один» (Plus-Minus One, PM1) является модификацией метода LSB. Данный метод меняет младший бит пикселя на бит сообщения

		посредством случайного уменьшения или увеличения значения пикселя на единицу в следующих случаях: – если значение пикселя четное, и бит секретного сообщения равен единице; – если значение пикселя нечетное, и бит секретного сообщения равен нулю.
QIM (Quantization Index Modulation)	Пространственное встраивание	Метод модуляции индекса квантования (Quantization Index Modulation, QIM) состоит в изменении значений пикселей изображения в зависимости от значений встраиваемых битов сообщения по формуле:
PVD (Pixel Value Differencing)	Пространственное встраивание	Метод разности значений пикселей (Pixel Value Differencing, PVD) основан на изменении разности между значениями соседних пикселей изображения.
SVD	Частотные преобразования	раскладываем блоки изображения на «узоры» и их «силы» (сингулярные значения) и слегка правим эти

		«силы» по ключу, чтобы закодировать биты.
ДКП	Частотные преобразования	Блочная ДКП: метка встраивается в среднечастотные коэффициенты (аддитивно, по QIM/STDM или spread-spectrum). На проверке тестируют отношения/косеты; удобно строить блочную карту tamper (semi-fragile аутентификация). Хорошо «дружит» с JPEG.
ДВП	Частотные преобразования	Многоуровневое вейвлет-представление: встраивание в выбранные поддиапазоны (LH/HL, иногда LL) аддитивно или по QIM/STDM, часто с HVS-маскированием. ДВП дает мультиразрешение и тонкую настройку между незаметностью и робастностью; популярен для робастных и semi-fragile ЦВЗ, а также в гибридах ДВП-SVD.

1.2 Современные подходы

Современный этап развития цифровых водяных знаков во многом связан с тем, что изменился характер угроз и сценариев использования изображений. Если ранее основными типовыми преобразованиями считались сжатие, пересохранение и простая постобработка, то сейчас изображения часто проходят через сложные цепочки автоматической обработки в сервисах, через

нейросетевые улучшения качества, а также через генеративное редактирование (дорисовку, замену объектов, вставку фрагментов). Кроме того, у практических систем выросли требования к масштабируемости: водяной знак должен работать в условиях массовой публикации, быть устойчивым к широкому набору преобразований и при этом обеспечивать контролируемую проверку – от подтверждения происхождения до локализации вмешательства. На этом фоне наиболее популярными становятся методы, которые опираются на обучаемые модели.

Наиболее заметным направлением стали нейросетевые (обучаемые) водяные знаки, которые строятся по принципу «encoder–decoder» (кодировщик – декодировщик). В таких системах модель обучается так, чтобы встраивать метку в изображение с минимальным ухудшением качества и затем извлекать ее после заранее заданного набора искажений. Ключевое отличие от классических методов состоит в том, что устойчивость здесь достигается не за счет ручного выбора коэффициентов и правил квантования, а за счет обучения на примерах: в процесс обучения включают типовые преобразования (сжатие, шум, изменение размера, фильтрацию) и заставляют модель сохранять возможность корректного извлечения.

В работе [25] показывается устройство одного из таких методов – EditGuard. Ключевая идея – совместить два типа водяных знаков в одном контейнере:

- глобальную (битовую) метку для подтверждения авторства (робастный компонент),
- пространственно – распределенную метку для локализации изменений (полу–хрупкий компонент).

Архитектура реализует последовательное встраивание и параллельное извлечение обоих каналов. Пространственный канал трактуется как задача извлечения скрытого «опорного» изображения (или его устойчивого

представления), чувствительного к локальным вмешательствам: расхождения между эталоном и извлеченным представлением индуцируют маску правок. Глобальный канал кодирует компактную подпись правообладателя и оптимизируется на робастность к типовым деградациям (сжатие, шум, умеренные геометрические преобразования).

Схема работы платформы EditGuard изображена на рисунке 1.1.

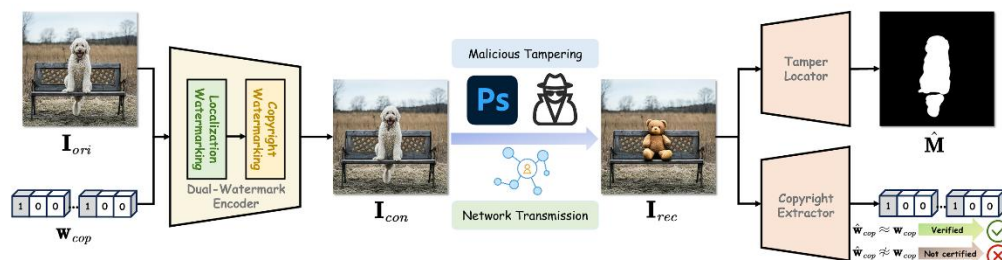


Рисунок 1.1 – Схема работы платформы EditGuard [25]

На рисунке 1.1 показано, как двойной кодер водяных знаков последовательно встраивает predetermined водяной знак локализации и водяной знак авторского права в исходное изображение, генерируя изображение–контейнер.

Итогом работы является единая проактивная схема, совмещающая подтверждение авторства, локализацию вмешательств и согласование с деградациями на этапе извлечения.

Впоследствии, в 2025 году, был выпущен инструмент [26] для повышения точности EditGuard. Новый инструмент OmniGuard позволяет снять ограничения по точности, гибкости и устойчивости, особенно при редактировании контента, сгенерированного искусственным интеллектом (AIGC).

Ключевой идеей данного инструмента являлась «гибридная схема»: проактивное встраивание дуального водяного знака. Локализационный знак подбирается адаптивно. Кроме того, в метод добавлен облегченный слой моделирования AIGC-правок, который помогает учитывать возможные изменения, возникающие при генеративном редактировании:

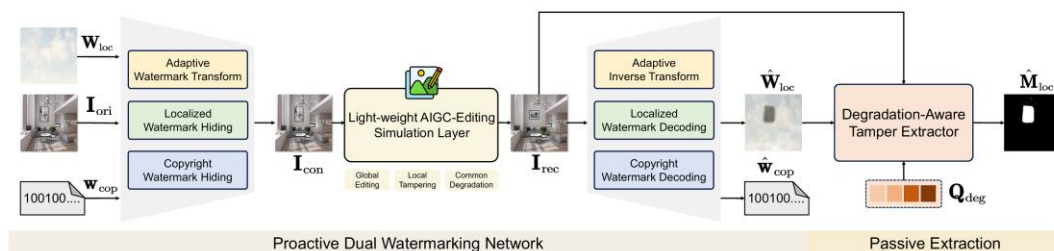


Рисунок 1.2 – Схема работы OmniGuard [26]

На рисунке 1.2 показана общая схема работы OmniGuard. Сначала применяется адаптивное прямое преобразование, с помощью которого исходное изображение связывается с локализационным водяным знаком. Благодаря этому водяной знак получает текстуру, согласованную с содержимым изображения, и может быть встроен менее заметно для зрителя.

Как итог OmniGuard превосходит передовые методы: формирует изображения с более высоким качеством и меньшим числом артефактов. Кроме того, метод обеспечивает более точную локализацию вмешательств и более надежное извлечение авторской битовой метки даже при сильных деградациях изображения и правках, выполненных с использованием генеративного искусственного интеллекта

В 2022 году впервые была опубликована концепция FaceSigns: нейронная полухрупкая схема встраивания ЦВЗ для аутентификации и противодействия DeepFake [27, 28, 29]. В этой статье была сформулирована задача проактивной аутентификации с комбинированным набором трансформаций и анализом вектора угроз.

Аутентификация медиа, несмотря на то что была долгосрочной проблемой, стала еще более сложной с появлением генеративных моделей, основанных на глубоком обучении. Методы синтеза медиа на основе глубоких нейронных сетей (DNN) [28, 29] позволили создавать высококачественные убедительные поддельные фотографии и видео. Одним из таких примеров синтетических видео являются Deepfakes, в которых лицо субъекта заменяется целевым лицом для

создания убедительно реалистичных кадров событий, которые никогда не происходили [30, 31]. Для решения вышеуказанных проблем была разработана система FaceSigns – полухрупкая система водяных знаков, основанная на глубоком обучении, которая встраивает восстанавливаемое сообщение как незаметное изменение в пиксели изображения.

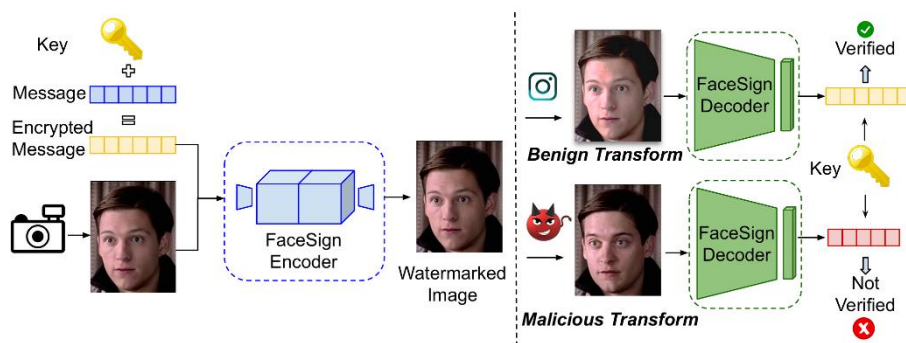


Рисунок 1.3 – Инфраструктура водяных знаков FaceSigns [27]

На рисунке 1.3 изображена инфраструктура водяных знаков: сеть кодировщиков встраивает секретное зашифрованное сообщение в заданное изображение в виде незаметного водяного знака, который устойчив к безопасным преобразованиям изображений и инструментам редактирования фотографий, но уязвим к вредоносным манипуляциям с изображениями, таким как Deepfakes.

В статье были представлены новые системы водяных знаков, основанные на глубоком обучении, которые позволяют сертифицировать целостность цифровых изображений и видео и надежно выявлять подделки. В ходе испытаний было продемонстрировано, что концепция FaceSigns создает более незаметные водяные знаки, чем предыдущие передовые модели.

В рамках современных угроз и сценариев распространения изображений цифровые водяные знаки эволюционировали от «точечных» алгоритмов к полноценным проактивным системам, в которых ключевые свойства метки формируются обучением. Нейросетевые схемы типа «encoder-decoder» позволяют одновременно контролировать качество контейнера и устойчивость

извлечения, поскольку устойчивость достигается за счет обучения на наборе реалистичных деградаций, а не за счет фиксированных правил внедрения

1.3 Метрики оценки качества встраивания ЦВЗ

Эффективность алгоритма цифрового водяного знака оценивается по нескольким группам показателей: качеству контейнерного изображения, надежности обнаружения или извлечения метки, емкости и точности локализации вмешательства. Ниже приведены основные метрики, которые далее могут использоваться при постановке экспериментов.

- Для задачи обнаружения метки используются доля ложных срабатываний (FPR) и доля пропусков (FNR):

$$FPR = \frac{FP}{FP+TN}; \quad (1)$$

$$FNR = \frac{FN}{FN+TP}, \quad (2)$$

где FP – ложноположительные решения;

TN – истинно отрицательные решения;

FN – ложноотрицательные решения;

TP – истинно положительные решения.

- Если водяной знак содержит битовое сообщение, качество извлечения можно оценивать через долю битовых ошибок:

$$BER = \frac{Nerr}{N}, \quad (3)$$

где $Nerr$ – число неверно восстановленных битов;

N – общее число битов сообщения.

Для шаблонной или распределенной метки также может применяться NCC , то есть нормализованная корреляция извлеченной метки с эталоном.

- Незаметность встраивания оценивается через искажение контейнерного изображения. Среднеквадратичная ошибка вычисляется как:

$$MSE = \frac{1}{mn} \times \sum_{i=1}^m \sum_{j=1}^n (I_{ij} - K_{ij})^2, \quad (4)$$

где I – исходное изображение;

K – контейнерное изображение; 19к слов

m и n – размеры изображений.

- Пиковое отношение сигнал/шум определяется как:

$$PSNR = 10 \times \lg\left(\frac{MAXI^2}{MSE}\right), \quad (5)$$

где $MAXI$ – максимальное значение яркости пикселя.

Дополнительно используется SSIM, который оценивает сохранение структуры изображения:

$$SSIM = \frac{(2\mu_x\mu_y + C_1) \times (2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1) \times (\sigma_x^2 + \sigma_y^2 + C_2)}, \quad (6)$$

где x – исходное изображение;

y – изображение после встраивания цифрового водяного знака;

μ_x, μ_y – средние значения яркости изображений;

σ_x^2, σ_y^2 – дисперсии изображений;

σ_{xy} – ковариация между изображениями,

C_1, C_2 – малые стабилизирующие константы, предотвращающие деление на ноль.

- Емкость характеризует объем данных, который переносит водяной знак.

В простейшем виде она задается как:

$$payload = \frac{B}{m \times n}, \quad (7)$$

где B – число встроенных битов;

$m \times n$ – число пикселей изображения. Для локализации вмешательств дополнительно могут применяться IoU, F1-score и mAP, поскольку результатом является не только факт обнаружения, но и маска измененной области:

$$IoU = \frac{M_{pred} \cap M_{gt}}{M_{pred} \cup M_{gt}}, \quad (8)$$

где M_{pred} – предсказанная маска измененной области;

M_{gt} – эталонная маска измененной области;

$M_{pred} \cap M_{gt}$ – область их пересечения;

$M_{pred} \cup M_{gt}$ – область их объединения.

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}, \quad (9)$$

$$Precision = \frac{TP}{TP + FP}, \quad (10)$$

$$Recall = \frac{TP}{TP + FN}, \quad (11)$$

где TP – число пикселей, которые действительно относятся к измененной области и были правильно найдены;

FP – число пикселей, ошибочно отнесенных к измененной области;

FN – число пикселей измененной области, которые метод не обнаружил.

$$mAp = \frac{1}{N} \times \sum_{i=1}^N AP_i, \quad (12)$$

где N – число изображений, классов;

AP_i – средняя точность для отдельного изображения, класса

В результате корректная оценка алгоритма ЦВЗ сводится к сопоставлению нескольких групп метрик. Такой подход позволяет описать компромисс между качеством контейнера, устойчивостью извлечения, емкостью и точностью локализации фальсификаций.

1.4 Атаки на цифровые водяные знаки

В контексте цифровых водяных знаков под атакой обычно понимают любое воздействие на изображение–контейнер, которое ухудшает обнаружение или извлечение метки, снижает ее доказательную силу либо позволяет обойти

проверку целостности. На практике атаки удобно рассматривать через их цель: одни направлены на «стирание» водяного знака, другие – на нарушение условий, при которых он корректно считывается, третьи – на подделку результата проверки и присваивание авторства.

Наиболее прямой класс – атаки на разрушение или ослабление метки (Removal attacks / Watermark removal). Их задача заключается в том, чтобы водяной знак перестал уверенно обнаруживаться, при этом само изображение оставалось визуально приемлемым. В эту группу обычно относят повторное сжатие и перекодирование (например, изменение параметров JPEG), добавление шума, сглаживание и фильтрацию, а также различные операции коррекции изображения – изменение контраста, яркости и гаммы. Все эти воздействия по-разному влияют на сигнал водяного знака, но общий эффект один: метка либо искажается настолько, что растет ошибка извлечения, либо ее статистические признаки становятся слишком слабыми для детектора. В более целенаправленном варианте атакующий может попытаться оценить структуру метки по изображению и затем вычистить ее или подавить, то есть перейти от «случайной деградации» к попытке удаления метки как сигнала.

Отдельно выделяют атаки рассинхронизации (Desynchronization attacks / Synchronization attacks). В отличие от разрушения метки как таковой, здесь цель часто состоит в том, чтобы нарушить согласование между тем, как метка была встроена, и тем, как система ожидает ее найти. Для многих робастных схем критично, чтобы сохранялась структура разбиения на области или корректно сопоставлялись координаты при извлечении. Поэтому поворот, масштабирование, сдвиг, отражение, обрезка и более сложные геометрические искажения способны резко ухудшить детектирование даже в тех случаях, когда «энергия» метки в изображении формально еще присутствует. Практически это один из самых эффективных способов обхода: изображение остается визуально

почти тем же, но детектор перестает извлекать метку, поскольку ожидаемый шаблон или геометрическая привязка нарушены.

Следующая крупная группа – атаки на подделку и присваивание (Forgery attacks / Ownership attacks). Здесь атакующий не обязательно стремится полностью уничтожить метку; вместо этого он пытается создать ситуацию, в которой можно предъявить ложное доказательство владения или подлинности. Это может выражаться в повторном маркировании уже помеченного изображения другой меткой, во внедрении собственной метки поверх чужой, либо в переносе (копировании) водяного знака на другой контент так, чтобы проверка ошибочно признала его легитимным. В таких сценариях критична не только стойкость самой метки, но и корректность протокола доказательства: какие ключи используются, как выбираются пороги, как трактуются конфликтующие метки и при каких условиях результат проверки считается юридически значимым.

Отдельно рассматривают коллюзионные атаки (Collusion attacks), которые становятся возможны, когда у атакующего есть несколько версий одного и того же контента с разными метками. В этом случае можно пытаться усреднять или агрегировать такие копии, чтобы ослабить индивидуальный сигнал метки и приблизиться к исходному изображению без водяного знака.

В современных условиях особенно значимыми стали также контентные атаки (Content editing attacks / Tampering attacks) и генеративное редактирование. В отличие от «технических» преобразований типа сжатия, такие вмешательства меняют смысл изображения: удаление объекта и дорисовка фона, вставка фрагмента, замена области, композиция нескольких источников. Для систем аутентификации и локализации такие воздействия являются особенно важными, так как одна из ключевых задач заключается в выявлении содержательной подмены. В случае нейросетевых водяных знаков появляется дополнительная проблема: атакующий может подбирать изменения с учетом поведения модели-

детектора. К таким воздействиям относятся едва заметные изменения, которые приводят к ухудшению извлечения метки, и обработка изображения с помощью моделей редактирования, целью которых является разрушение статистических признаков водяного знака.

Таким образом, атаки на цифровые водяные знаки можно рассматривать как набор стратегий с разными целями: от прямого уничтожения встроенной метки до более изящного обхода проверки через фальсификацию доказательства или генеративное редактирование изображения. В прикладных задачах их необходимо явно связывать с целями системы: робастные метки должны устойчиво переживать типовую обработку, полухрупкие – выдерживать допустимые преобразования и реагировать на содержательные вмешательства, а локализирующие схемы должны быть способны дополнительно показывать область внесенных изменений, особенно в сценариях AIGC-редактирования.

1.5 Выводы по литературному обзору

Проведенный анализ литературы показывает, что цифровые водяные знаки все еще являются одним из возможных инструментов защиты и проверки изображений в условиях автоматической обработки, нейросетевого улучшения и генеративного редактирования. Для распознавания сгенерированных объектов особенно важны схемы, которые позволяют локализовать область вмешательства.

Классические методы по-прежнему составляют основу данной предметной области. Пространственные подходы отличаются простотой реализации и позволяют встраивать сравнительно большой объем данных, однако обычно хуже переносят сжатие и последующую обработку изображения. Методы, работающие в области преобразований, как правило, устойчивее к типовым изменениям и дают больше возможностей для настройки баланса между визуальным качеством и надежностью извлечения. При практическом применении важную роль играет и способ извлечения метки, поскольку от него

зависит, можно ли проверять изображения массово и без доступа к исходному файлу.

Современные исследования все чаще используют методы глубинного обучения. Здесь свойства водяного знака формируются обучением на примерах и моделируемых искажениях. Это особенно важно в условиях генеративных вмешательств, когда нужно выявить участки изменений. Наиболее перспективными для темы распознавания сгенерированных объектов являются проактивные решения, которые совмещают устойчивую подпись авторства и компонент для локализации правок. Поэтому в следующем разделе рассматривается OmniGuard как базовый метод, совмещающий авторскую битовую метку и компонент локализации правок.

2. Базовый метод встраивания с поддержкой распознавания сгенерированных изображений

Во второй главе обосновывается выбор OmniGuard как базового метода для дальнейшего исследования. Сначала сопоставляются классические и современные подходы, затем кратко описывается принцип работы OmniGuard, после чего фиксируются выполненные изменения, необходимые для воспроизводимого запуска существующего решения.

Дополнительное требование связано с тем, что вмешательство может быть не только грубым редактированием, но и добавлением реалистичного сгенерированного объекта. В таком сценарии визуальная незаметность изменения для человека не должна означать невозможность его последующего обнаружения алгоритмом.

С практической точки зрения предпочтение отдается слепым и полуслепым схемам, так как они лучше согласуются с интернет–сценарием проверки изображений: у проверяющей стороны обычно есть только изображение–контейнер, ключ или детектор, но нет исходного файла.

2.1 Выбор базового метода

В ходе исследования в качестве базовых ориентиров были рассмотрены уже описанные в литературном обзоре методы LSB, PVD и метод на основе ДКП. Они нужны для задания нижнего уровня сравнения при дальнейших экспериментах.

LSB и PVD относятся к пространственным подходам. Их сильная сторона состоит в простоте реализации и сравнительно высокой емкости, однако устойчивость к постобработке и сложному редактированию ограничена. Поэтому такие методы целесообразно использовать как контрольные базовые линии.

Метод на основе ДКП работает в частотной области и лучше согласуется с JPEG–сжатием. Он подходит для сравнения с OmniGuard как классический

частотный подход, но его правила встраивания и извлечения обычно задаются заранее и хуже адаптируются к генеративному редактированию.

Сводное сравнение базовых методов и современных нейросетевых схем приведено в таблице 2.1.

Таблица 2.1 – Краткое сравнение базовых методов и современных нейросетевых схем

Метод	Класс	Сильная сторона	Ограничение	Роль в работе
LSB	Пространственный	Простота и высокая емкость	Слабая устойчивость к постобработке	Базовый ориентир для простого встраивания
PVD	Пространственный адаптивный	Лучший баланс незаметности и емкости	Ограниченная устойчивость	Базовый ориентир для адаптивного пространственного встраивания
ДКП	Частотный	Лучше переносит JPEG и удобен для полухрупкой аутентификации	Жесткие правила и ограниченная гибкость	Базовый ориентир для частотного подхода
EditGuard	Нейросетевой проактивный	Одновременная локализация правок и защита авторства	Зависимость качества маски от восстановления локализационного знака	Непосредственный предшественник OmniGuard
OmniGuard	Гибридный нейросетевой	Лучшая гибкость, устойчивость и качество контейнера	Более сложная архитектура и требования к реализации	Основной объект практической адаптации

Сопоставление показывает, что простые схемы удобны как контрольные ориентиры, но для задачи локализации сгенерированных объектов в реалистичных условиях их возможностей недостаточно. Поэтому в качестве базовой платформы выбран OmniGuard.

Практически значимые различия между EditGuard и OmniGuard приведены в таблице 2.2.

Таблица 2.2 – Практически значимые различия между EditGuard и OmniGuard

Признак	EditGuard	OmniGuard
Локализационный знак	Предопределенный и фактически фиксированный	Адаптивный и более гибкий
Извлечение маски	По разности между заданным и восстановленным знаком	Через отдельный пассивный модуль извлечения маски
Устойчивость к деградациям	Хорошая, но чувствительная к качеству восстановления знака	Повышенная за счет деградационно–осведомленного извлечения
Поддержка AIGC–редактирования	Есть, но с ограничениями	Расширена за счет симуляции глобального и локального редактирования
Качество контейнера	Ниже из-за жесткого компромисса между локализацией и качеством	Выше благодаря адаптивному преобразованию знака
Практический итог	Рабочая универсальная база	Более сильный кандидат для дальнейших экспериментов

2.2 OmniGuard

OmniGuard представляет собой гибридный метод цифрового водяного маркирования изображений, предназначенный для локализации манипуляций и извлечения авторской метки. Его логика развивает EditGuard, но добавляет механизмы, которые уменьшают зависимость результата от фиксированного локализационного водяного знака.

Метод объединяет проактивное встраивание и пассивное слепое извлечение [6, 7, 8]. Проактивное встраивание означает, что метка добавляется в изображение до возможного распространения или редактирования. Пассивное слепое извлечение означает получение карты вмешательства без доступа к

исходному изображению, только по проверяемому контейнеру и модели извлечения. Такой режим лучше соответствует реальным условиям проверки изображений.

Идейно алгоритм OmniGuard включает три основных элемента. Первый элемент – адаптивное преобразование локализационного водяного знака. В отличие от фиксированного локализационного шаблона, адаптивный знак согласуется с содержимым изображения. Это снижает видимость внедрения и помогает сохранить качество контейнера.

Второй элемент – встраивание двух меток: локализационной и авторской. Локализационная метка используется для поиска области вмешательства, а авторская битовая метка нужна для подтверждения принадлежности изображения. Совмещение этих компонентов позволяет решать сразу две задачи: проверку происхождения и выявление зоны правки.

Третий элемент – деградационно-осведомленный модуль извлечения маски. Под этим понимается модуль, который учитывает возможные искажения изображения после редактирования и формирует карту вероятной области вмешательства не только по восстановленному знаку, но и по самому проверяемому изображению. За счет этого локализация меньше зависит от идеального восстановления локализационной метки.

Важной частью OmniGuard является симуляция AIGC-редактирования. Она используется для повышения устойчивости извлечения авторской подписи в условиях глобального и локального генеративного редактирования. Такой механизм особенно важен для темы распознавания сгенерированных объектов, поскольку изменения могут выглядеть естественно и не выделяться визуально.

Общая логика применения цифрового водяного знака для защиты изображения и поиска правок показана на рисунке 2.1.



Рисунок 2.1 – Общая цепочка применения цифрового водяного знака для защиты изображения и поиска правок

На рисунке 2.1 показан общий сценарий: исходное изображение получает водяной знак, затем может подвергаться редактированию, после чего модуль извлечения формирует карту вмешательства. Для OmniGuard принципиально важно, что проверка выполняется без необходимости иметь исходный файл на стороне проверяющего.

Принципиальное различие логики формирования маски в EditGuard и OmniGuard показано на рисунке 2.2.

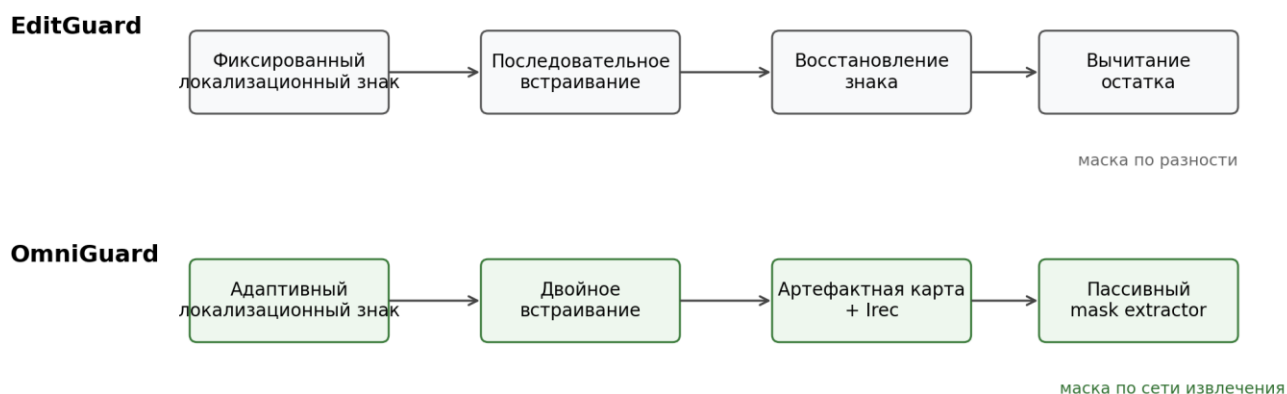


Рисунок 2.2 – Упрощенное сравнение логики формирования маски в EditGuard и OmniGuard: Irec – восстановленное изображение, mask extractor – модуль извлечения маски

На рисунке 2.2 выделены элементы архитектуры, за счет которых OmniGuard снижает зависимость локализации от фиксированного локализационного знака. В EditGuard маска строится в основном по различию между заданным и восстановленным знаком. В OmniGuard используется отдельный модуль извлечения, который дополнительно учитывает деградации и само проверяемое изображение.

Количественное превосходство OmniGuard над EditGuard отражено в статье [26]. Авторы указывают выигрыш 4,25 дБ по PSNR контейнерного изображения, 20,7 % по F1–score в шумных условиях и 14,8 % по средней битовой точности извлечения авторской метки. Для дальнейшего исследования это означает, что OmniGuard является более сильной базой, чем EditGuard.

2.3 Актуализация существующего решения OmniGuard

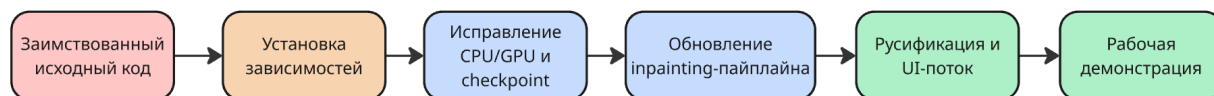
После изучения статей следующим этапом исследования стала работа с заимствованной исследовательской реализацией OmniGuard. В качестве базы использовался опубликованный исследовательский прототип. В исходном виде он не мог быть использован как воспроизводимый инструмент: запуск сопровождался ошибками зависимостей, проблемами загрузки моделей и некорректной работой пользовательского интерфейса.

Поэтому выполненная работа заключалась не в изменении архитектуры OmniGuard, а в актуализации существующего решения под локальный запуск. Это было необходимо для того, чтобы далее можно было проводить эксперименты с базовым методом и сравнивать его с последующей модификацией.

Таблица 2.3 – Основные проблемы исходной реализации и принятые решения

Проблема	Как проявлялась	Что было сделано	Практический эффект
Зависимости среды	Приложение не стартовало из-за отсутствующих пакетов и неконсистентных версий	Собрано отдельное окружение и согласован базовый набор зависимостей	Исходный код стал воспроизводимо запускаться
Жесткая привязка к CUDA	Часть модулей предполагала наличие графического процессора	Переработано определение устройства и загрузка моделей для CPU и GPU	Код удалось запускать без обязательной видеокарты
Старый формат checkpoint	Загрузка весов ломалась на новых версиях PyTorch	Исправлена работа с checkpoint-файлами и отображением весов на вычислительное устройство	Весовые файлы стали корректно читаться
Inpainting-пайплайн	Модель редактирования не загружалась или работала нестабильно	Обновлен механизм загрузки модели редактирования и снижена нагрузка на CPU	Шаг редактирования изображения начал выполняться стабильно
Пользовательский интерфейс	Логика второго и третьего шагов не совпадала	Исправлен поток данных между этапами и русифицированы ключевые элементы	Интерфейс стал пригоден для демонстрации
Жестко заданные пути	Программа пыталась сохранять результаты в несуществующие системные каталоги	Пути заменены на локальные рабочие директории проекта	Третий шаг локализации перестал завершаться ошибкой

Суммарно эта часть работы превратила исходный код из исследовательского артефакта в демонстрационный инструмент, пригодный для дальнейших экспериментов. Схема самой адаптации приведена на рисунке 2.3.



Цель адаптации: перевести исследовательский прототип в локально воспроизводимый инструмент

Рисунок 2.3 – Основные этапы инженерной адаптации заимствованной реализации OmniGuard

Выполненные изменения целесообразно описывать не через строки кода, а через их влияние на работоспособность базового метода. Первое направление касалось инициализации модели редактирования изображения: был обеспечен явный выбор вычислительного устройства и согласован способ загрузки моделей.

Второе направление касалось передачи данных между этапами интерфейса. Результат локального редактирования был приведен к формату, пригодному для последующей локализации, а обработка пустого или некорректного ввода была отделена от штатного сценария.

Третье направление касалось служебных путей и промежуточных результатов. Жестко заданные каталоги были заменены на локальные рабочие директории проекта, которые создаются и используются внутри текущей файловой структуры.

Эти изменения не меняют математическую идею OmniGuard, но делают возможным его воспроизводимое тестирование. Без такого этапа базовый метод не доходил бы до интерпретируемого результата и не мог бы использоваться как точка сравнения.

После актуализации получен рабочий демонстрационный сценарий. На первом этапе пользователь загружает исходное изображение и запускает встраивание цифрового водяного знака. На втором этапе задается область редактирования и выполняется моделирование локального вмешательства. На третьем этапе изображение подается в модуль извлечения, который формирует карту вероятной области правки.

Маска, возвращаемая на третьем этапе, интерпретируется как карта отклика модели. Для ее последующей количественной оценки может использоваться сравнение с эталонной областью редактирования через IoU, F1-score или mAP: итог зависит не только от факта обнаружения вмешательства, но и от совпадения найденной области с реальной областью правки.

В текущем демонстрационном сценарии важно учитывать ограничение: если область редактирования заметна визуально, такой пример не полностью отражает задачу поиска реалистичных сгенерированных объектов. Поэтому дальнейшие эксперименты должны включать более незаметные локальные вставки и генеративные правки.

2.4 Эффективность OmniGuard

Эффективность OmniGuard в рамках данной работы должна оцениваться по тем же группам показателей, которые выделены в литературном обзоре: незаметность встраивания, емкость, точность локализации фальсификаций и устойчивость к искажающим воздействиям. На текущем этапе получен не полный набор сравнительных результатов, а рабочая база для их получения.

Сильная сторона OmniGuard состоит в сочетании авторской битовой метки и локализационного компонента. При этом для дальнейшей работы существенны и слабые места базового метода: зависимость качества результата от типа редактирования, необходимость проверки на реалистичных сгенерированных объектах и потребность в устойчивой оценке при санкционированных искажениях.

Следующим этапом станет проверка базового OmniGuard на выбранных наборах данных и сопоставление полученных результатов с EditGuard, LSB и методом на основе ДКП. Именно выявленные ограничения базового метода будут использованы как основание для собственной модификации в третьей главе.

3. Усовершенствованный метод встраивания с поддержкой распознавания сгенерированных изображений

В третьей главе описываются алгоритмические изменения, внесенные относительно исходного OmniGuard. Исходный прототип был ориентирован главным образом на восстановление скрытого водяного знака и построение маски вмешательства. Эта идея была сохранена как базовая, поскольку именно она позволяет связывать нарушение скрытого сигнала с областью редактирования. При этом для практической проверки изображения одной маски недостаточно: требуется понимать, какие данные были встроены, относятся ли они к ожидаемому идентификатору и насколько устойчиво они восстанавливаются после типовых искажений.

Поэтому усовершенствование было выполнено не как замена нейросетевого ядра OmniGuard, а как расширение его алгоритмического контура. Поверх исходной логики были добавлены проверяемые служебные данные, привязка к идентификатору документа, контроль целостности, коррекция ошибок, расширенное построение карты вмешательства и автоматизированный экспериментальный контур. В результате исходный демонстрационный сценарий был преобразован в систему, которая позволяет не только показать локализацию правки на одном изображении, но и проводить воспроизводимое сравнение методов на наборе изображений.

3.1 Проверяемая и устойчивая служебная битовая метка

В исходном прототипе OmniGuard использовалась 100-битная строка, по которой можно было оценивать долю битовых ошибок. Для прикладной проверки этого недостаточно: восстановленная последовательность должна иметь понятную структуру, а отдельные ошибки извлечения не должны сразу приводить к потере всей записи. Поэтому в усовершенствованном варианте 100-

битный канал используется для переноса компактной служебной битовой метки с коррекцией ошибок.

Служебная метка не является самостоятельным видимым знаком на изображении. Она встраивается внутрь ЦВЗ как робастный битовый компонент и дополняет локализационный водяной знак. Ее назначение состоит в том, чтобы проверить корректность извлечения данных и связать результат проверки с конкретным защищаемым изображением. При этом основным признаком вмешательства по-прежнему определяется не разрушением хеша, а локализационной картой нарушения водяного знака.

Структура метки сначала формируется как 56 информационных битов. В нее входят версия формата, время создания, короткий хеш идентификатора изображения, случайное значение и НМАС-тег. После этого каждые 4 информационных бита кодируются кодом Хэмминга (7,4). В результате 14 блоков по 4 бита преобразуются в 98 бит, к которым добавляются 2 служебных бита заполнения. Итоговая длина становится равной 100 битам и остается согласованной с исходным каналом модели.

Таблица 3.1 – Структура встроенной служебной информации усовершенствованного метода

Поле	Размер	Назначение
Версия	4 бита	Версия формата служебных данных
Время создания	20 бит	Время создания в часах от фиксированной эпохи
Хеш идентификатора	16 бит	Короткий хэш от идентификатора документа
Случайное значение	8 бит	Случайное значение для уникальности записи
НМАС-тег	8 бит	НМАС–тег для проверки целостности

Механизм исправления ошибок выражается в декодировании каждого 7-битного блока по синдрому кода Хэмминга. Если в блоке поврежден один бит, синдром указывает его позицию, после чего бит исправляется и из блока извлекаются исходные 4 информационных бита. Так единичные ошибки, возникающие после сжатия, шума или другой допустимой обработки, не приводят к немедленной потере служебной метки.

НМАС-тег используется после коррекции ошибок как контроль достоверности восстановленной записи. Если ошибок слишком много и код Хэмминга уже не может восстановить корректные данные, НМАС-проверка не проходит. За счет этого служебный компонент остается ориентированным на полухрупкую проверку: допустимые искажения частично компенсируются кодом исправления ошибок, а существенное повреждение приводит к недостоверному результату.

Хеш идентификатора изображения занимает только часть служебной метки и не заменяет сам механизм ЦВЗ. Он нужен для прикладного сопоставления результата с ожидаемым изображением или записью. Поскольку значение хеша извлекается уже после коррекции ошибок, его использование не превращает всю схему в хрупкую: устойчивость обеспечивается кодированием битовой метки и локализационным компонентом водяного знака.

3.2 Двухкомпонентная структура цифрового водяного знака

В исходном OmniGuard применялись две метки: локализационная метка для поиска области вмешательства и авторская битовая метка для подтверждения принадлежности изображения. Локализационная часть фактически является водяным знаком, чувствительным к вмешательствам: если участок изображения был изменен, скрытый защитный сигнал в этой области восстанавливается хуже, и по этому отклонению можно построить маску подозрительной зоны.

Ограничение исходного варианта состоит в том, что авторская битовая метка не была оформлена как самостоятельная проверяемая служебная запись.

Она позволяла оценивать только факт восстановления битовой последовательности и не содержала прикладной структуры для проверки соответствия конкретному изображению и контроля целостности.

В усовершенствованном варианте сохранена исходная идея двух компонентов, но изменена роль второго компонента. Первый компонент остается локализационным и отвечает за построение карты вмешательства. Вторым компонент преобразован в служебную битовую метку: он содержит хеш идентификатора изображения, время создания, случайное значение, HMAC-тег и данные, защищенные кодом Хэмминга.

Служебная битовая метка выполняет вспомогательную проверочную функцию. Она позволяет оценить, насколько устойчиво встроенные данные восстанавливаются после допустимой обработки изображения. В прикладном сценарии это может использоваться для проверки опубликованного изображения или результата повторной загрузки в сервис, где файл мог быть сжат или пересохранен.

Важное отличие от исходного метода состоит в разделении задач между слоями. Локализационный слой остается чувствительным к пространственным изменениям и используется для построения карты вмешательства. Слой служебных данных работает как служебный канал, который должен сохраниться после допустимых искажений и подтвердить данные изображения. Такое разделение снижает зависимость всей системы от одного сигнала: нарушение локальной карты и повреждение встроенных служебных данных могут анализироваться отдельно.

3.3 Проблема построения информативной карты вмешательства

В исходном демонстрационном сценарии результат локализации был представлен как маска, построенная по восстановленному водяному знаку. Такой подход отражает основную идею OmniGuard, но в прикладных условиях карта может быть шумной, особенно если изображение было сжато, изменено по

яркости или отредактировано вне встроенного сценария локальной дорисовки. Кроме того, одна только карта нарушения не всегда отделяет реальную правку от фоновых изменений, вызванных преобразованием всего изображения.

Для повышения информативности результата были введены три режима построения карты. Первый режим использует карту нарушения водяного знака: она показывает, где восстановленный локализационный сигнал сильнее всего отличается от эталонного. Второй режим использует опорную карту, то есть сравнение анализируемого изображения с защищенной опорной версией. Третий режим формирует гибридную карту, объединяющую оба источника информации.

Для повышения читаемости карт добавлена процентильная нормализация и сглаживание. Нормализация уменьшает влияние отдельных выбросов и делает значения карты вмешательств удобнее для последующего порогового выделения. После этого карта может переводиться в бинарную маску. При наличии опорной применяется порог Оцу и морфологическая обработка: открытие убирает мелкий шум, закрытие заполняет небольшие разрывы, а расширение немного увеличивает найденную область.

Польза этой доработки состоит в том, что результат становится пригоднее не только для визуального просмотра, но и для количественной оценки. Бинарную маску можно сравнивать с эталонной областью изменения через IoU, F1-score или mAP. Тем самым локализация перестает быть только демонстрационным изображением и становится частью измеримого экспериментального контура.

3.4 Проблема ограниченности исходного демонстрационного сценария

Исходный проект был ближе к демонстрационному режиму одного изображения: пользователь загружал одно изображение, выполнял встроенное редактирование и получал маску. Такой формат удобен для демонстрации идеи, но недостаточен для полноценного эксперимента. Для оценки метода нужны

пакетная обработка изображений, атаки, сравнение с базовыми методами и единая система метрик.

В усовершенствованной версии был добавлен новый способ анализа. Он извлекает 100 бит встроенной служебной информации, декодирует код Хэмминга, проверяет HMAC, проверяет document_id, восстанавливает карту нарушения водяного знака, при наличии опорной версии строит reference-map, формирует hybrid heatmap, получает бинарную маску и сохраняет технический отчет. Также добавлен сценарий внешнего редактирования: изображение можно защитить, изменить во внешнем редакторе, загрузить обратно вместе с защищенной reference-версией и выполнить анализ.

Отдельно был реализован тестовый контур. Для каждого изображения выполняется встраивание ЦВЗ, применяется заданное искажение, извлекается встроенная служебная информация и считаются метрики. В контур включены JPEG-сжатие с разными параметрами, шумы, кадрирование, изменение яркости, медианный фильтр и уменьшение с последующим восстановлением размера. Кроме того, добавлены базовые методы LSB и ДКП, что позволяет сравнивать усовершенствованный OmniGuard не только с самим собой, но и с классическими подходами к цифровому водяному маркированию.

Еще одно важное отличие состоит в унификации обработки. Основная логика сведена в единый движок, который используется для защиты, редактирования, анализа, пакетного тестирования и сравнения методов. Это уменьшает риск того, что разные режимы программы будут считать метрики по-разному. Для эксперимента такая унификация важна, потому что все методы и атаки проходят через единый порядок обработки и результаты сохраняются в одном формате.

Таблица 3.2 – Отличия исходного и усовершенствованного алгоритма

Компонент	Исходный OmniGuard	Усовершенствованный вариант
-----------	--------------------	-----------------------------

Назначение служебных данных	Техническая битовая строка без прикладной структуры	Структурированные служебные данные с версией, временем, document_hash, nonce и HMAC
Связь с документом	Не предусмотрена	Проверка соответствия ожидаемому document_id
Устойчивость встроенных служебных данных	Нет отдельного слоя коррекции ошибок	Код Хэмминга (7,4) и учет исправленных ошибок
Локализация правок	Маска по восстановленному водяному знаку	Watermark-map, reference-map и hybrid heatmap с нормализацией
Экспериментальная проверка	Демонстрация на одном изображении	Пакетный benchmark, атаки, метрики и сравнение с LSB и ДКП

В результате основное отличие усовершенствованного метода состоит не в косметической переработке исходного прототипа, а в расширении его алгоритмических возможностей. Исходный OmniGuard решал задачу локализации вмешательства через нарушение скрытого водяного знака. Усовершенствованный вариант сохраняет эту часть, но дополнительно обеспечивает проверку служебных данных, связь с идентификатором изображения, устойчивость встроенных служебных данных к части ошибок и возможность пакетного сравнения с другими методами.

4. Экспериментальный стенд

Для проверки предложенного усовершенствованного метода был разработан экспериментальный стенд, позволяющий выполнять полный цикл работы с изображением: загрузку исходного файла, встраивание цифрового водяного знака, моделирование редактирования, анализ измененного изображения и расчет метрик. Стенд используется как инструмент для воспроизводимой проверки метода на наборах изображений.

Исходный код программного модуля, реализующего предложенную схему встраивания и экспериментальный стенд для ее проверки, размещен в приложении А.

Основная задача стенда состоит в том, чтобы объединить в одном программном контуре несколько режимов работы: защиту изображения, проверку встроенной служебной информации, построение карты изменений, сравнение разных методов встраивания и пакетный запуск экспериментов. Это важно, поскольку ручная проверка отдельных изображений не позволяет надежно оценить устойчивость метода. Для полноценного эксперимента требуется одинаковый порядок обработки изображений, единый набор атак и сохранение результатов в удобном для дальнейшего анализа формате.

Разработанный стенд основан на идее OmniGuard, но расширяет исходный исследовательский прототип прикладными средствами проверки. В частности, система поддерживает работу со встроенной служебной битовой меткой, проверку целостности, учет исправленных битовых ошибок, построение гибридной карты изменений и сохранение отчетов по результатам экспериментов (рисунок 4.1).

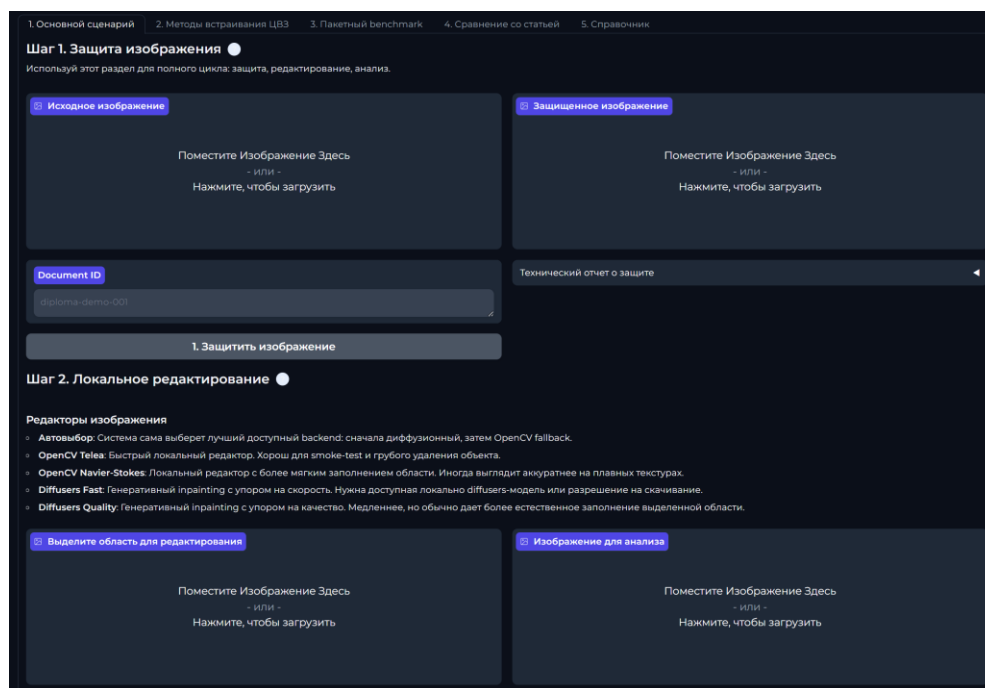


Рисунок 4.1 – Общий вид главной вкладки экспериментального стенда

4.1 Общая логика работы алгоритма в стенде

Работа стенда строится как последовательность нескольких этапов. Сначала пользователь загружает исходное изображение. Затем система формирует защищенную версию изображения, в которую встраиваются два типа информации: локальный водяной знак для последующей локализации вмешательства и служебная битовая метка для проверки встроенных данных.

После защиты изображение может быть подвергнуто редактированию. В демонстрационном режиме редактирование задается внутри интерфейса, а в экспериментальном режиме могут применяться заранее заданные воздействия: сжатие, шум, изменение яркости, кадрирование, фильтрация и масштабирование. Такой подход позволяет проверять метод не только на одном примере, но и в условиях, приближенных к типовой обработке изображений.

На этапе анализа система извлекает встроенную служебную информацию, проверяет ее корректность и строит карту подозрительности. Карта подозрительности показывает, какие области изображения с наибольшей вероятностью были изменены. Затем эта карта может быть преобразована в

бинарную маску, пригодную для сравнения с эталонной областью редактирования.

В стенде предусмотрены два режима локализации. Первый режим соответствует базовой логике OmniGuard и использует нарушение локального водяного знака. Второй режим использует гибридную карту, где учитывается не только повреждение водяного знака, но и отличие анализируемого изображения от защищенной опорной версии. Благодаря этому можно сравнивать исходный и усовершенствованный варианты в одинаковых условиях.

4.2 Основной пользовательский сценарий

Основной сценарий предназначен для демонстрации работы метода на одном изображении. Пользователь загружает файл изображения и при необходимости указывает идентификатор документа. После этого запускается защита изображения: система встраивает цифровой водяной знак и формирует защищенную версию файла (рисунок 4.2).

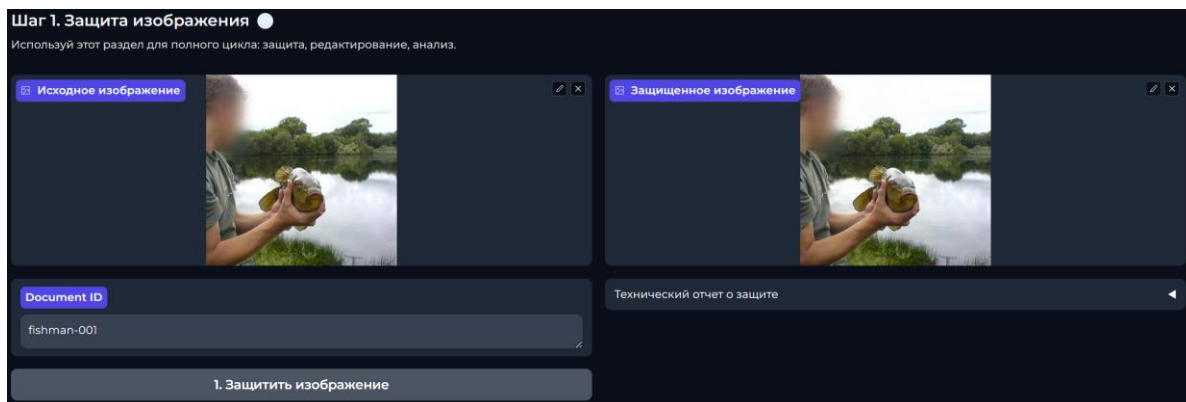


Рисунок 4.2 – Защита изображения

Далее пользователь задает или моделирует область редактирования, пишет промпт для генеративной модели и получает результат (рисунок 4.3).

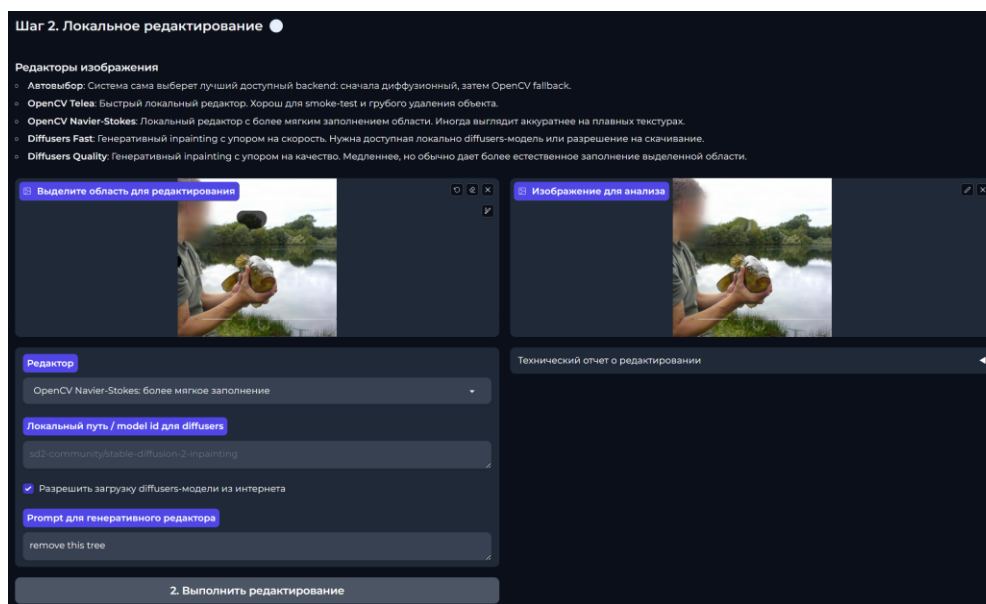


Рисунок 4.3 – Генеративное изменение изображения

После выполнения редактирования изображение передается на анализ. В результате интерфейс отображает защищенное изображение, измененное изображение, карту подозрительности, бинарную маску и технические показатели проверки. Такой сценарий удобен для наглядной демонстрации: можно проследить весь путь от исходного изображения до найденной области вмешательства (Рисунок 4.4).

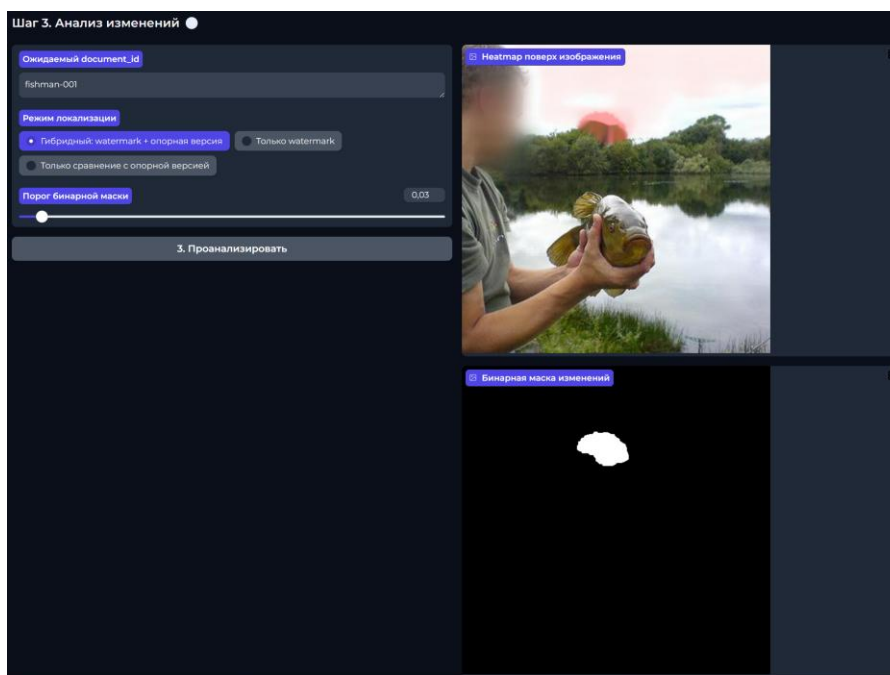


Рисунок 4.4 – Отображение маски изменений

Отдельное значение имеет поле идентификатора документа. Оно используется не как открытая надпись на изображении, а как внешний идентификатор, с которым сопоставляется встроенная служебная информация. Это позволяет проверить, относится ли анализируемое изображение к ожидаемой записи.

4.3 Режим сравнения методов

В стенде предусмотрена отдельная вкладка для сравнения методов встраивания цифровых водяных знаков. В ней сопоставляются усовершенствованный OmniGuard, метод замены наименее значимых битов и метод на основе дискретного косинусного преобразования. Для каждого метода система формирует изображение с водяным знаком и рассчитывает показатели качества контейнера и восстановления встроенной информации (рисунок 4.5).

Сравнение базовых методов

Раздел показывает результат встраивания, базовые метрики и технические параметры выбранного метода.

Изображение



Изображение с ЦВЗ



Payload / document_id

diploma-demo-001

Метод встраивания

☐ OmniGuard: нейросетевой ЦВЗ + payload

☒ LSB: младший бит пикселя

☐ DCT: коэффициенты ДКП 8x8

Встроить ЦВЗ

Метрики метода

Метрика	Значение	Как читать
PSNR	88.958282	Чем выше, тем менее заметен ЦВЗ
SSIM	1	Чем ближе к 1, тем больше структурное сходство
bpp	0.000488	Сколько бит ЦВЗ приходится на один пиксель
BER без атаки	0	Доля ошибочных бит сразу после извлечения
Встроено бит	128	Размер встроенного payload

Рисунок 4.5 – Демонстрация режима сравнения методов встраивания ЦВЗ

Этот режим нужен для предварительной проверки и демонстрации различий между подходами. Простые методы, такие как LSB, обычно дают

высокую визуальную незаметность, но хуже сохраняют встроенные данные после обработки изображения. Метод на основе ДКП является более устойчивым частотным подходом. OmniGuard проверяется как более сложная нейросетевая схема, ориентированная на устойчивое извлечение встроенной информации и локализацию вмешательств.

4.4 Пакетная экспериментальная проверка

Для проведения массовых экспериментов в стенд добавлен режим пакетной обработки. Он позволяет обрабатывать не одно изображение, а набор файлов. Пользователь может загрузить несколько изображений или указать папку с данными. После запуска система последовательно выполняет встраивание цифрового водяного знака, применяет выбранные воздействия, извлекает встроенную служебную информацию и сохраняет результаты.

Результатом пакетного тестового контура является отчет в формате CSV. Такой отчет содержит значения метрик по каждому изображению, методу и типу воздействия. Это делает эксперимент воспроизводимым: результаты можно пересчитать, агрегировать и использовать для построения таблиц в экспериментальном разделе.

Пакетный режим особенно важен для данной работы, поскольку итоговое сравнение проводится на большом наборе изображений (рисунок 4.6). Без такого режима проверка была бы сведена к отдельным демонстрационным примерам, что не позволило бы сделать обоснованный вывод об устойчивости метода.

Эксперимент на датасете

Подходит для 50-100 изображений. OmniGuard может работать дольше LSB/DCT; для быстрого черного прогона его можно снять в списке методов.

Пул изображений

000001.jpg 30.5 KB ↓

000002.jpg 47.4 KB ↓

000003.jpg 33.7 KB ↓

000004.jpg 63.2 KB ↓

000005.jpg 49.8 KB ↓

Или путь к папке с изображениями

C:\Users\MI\Downloads\dataset

Payload / document_id

diploma-demo-001

Методы для сравнения

✓ OmniGuard: нейросетевой ЦВЗ + payload

✓ LSB: младший бит пикселя

✓ DCT: коэффициенты ДКП 8x8

Запустить пакетный benchmark

Результаты benchmark

Изображение	Метод	Атака	PSNR	SSIM	bpp	BER
000001.jpg	omniguard	no_attack	40.789148	0.999662	0.000381	0
000001.jpg	omniguard	jpeg_q100	40.789148	0.999662	0.000381	0
000001.jpg	omniguard	jpeg_q90	40.789148	0.999662	0.000381	0
000001.jpg	omniguard	jpeg_q50	40.789148	0.999662	0.000381	0
000001.jpg	omniguard	gaussian_noise_0.01	40.789148	0.999662	0.000381	0
000001.jpg	omniguard	salt_pepper	40.789148	0.999662	0.000381	0.05
000001.jpg	omniguard	random_crop_10	40.789148	0.999662	0.000381	0
000001.jpg	omniguard	brightness_plus_15	40.789148	0.999662	0.000381	0.02
000001.jpg	omniguard	brightness_minus_15	40.789148	0.999662	0.000381	0
000001.jpg	omniguard	brightness_mul_1.5	40.789148	0.999662	0.000381	0.11
000001.jpg	omniguard	brightness_mul_0.8	40.789148	0.999662	0.000381	0
000001.jpg	omniguard	median_3x3	40.789148	0.999662	0.000381	0

Путь к CSV-отчету

C:\Users\MI\Downloads\OmniGuard-main\OmniGuard-main\untime\requirement_benchmarks\run_20260509T162923Z\requirement_benchmark.csv

Рисунок 4.6 – Эксперименты на датасете

4.5 Режим сравнения с исходным OmniGuard

Отдельный режим стенда предназначен для сопоставления исходной логики OmniGuard и усовершенствованного варианта. В этом режиме оба подхода проверяются в одинаковых условиях: на одном и том же изображении, с одной и той же областью вмешательства и с одинаковыми деградациями. Такой порядок нужен для корректного сравнения, поскольку различие в результатах должно быть связано именно с алгоритмом, а не с разными входными данными.

После запуска система показывает защищенное изображение, атакованное изображение, эталонную маску атаки, карту подозрительности и бинарную маску для исходного режима, а также соответствующие результаты для улучшенного режима. Дополнительно выводится таблица метрик по каждому изображению и сводная таблица средних значений. Отчеты сохраняются в форматах JSON и CSV.

Этот режим является наиболее важным для последующего сравнения с результатами статьи OmniGuard, поскольку он позволяет проверять исходную и улучшенную версии по единому протоколу (рисунок 4.7 и рисунок 4.8).

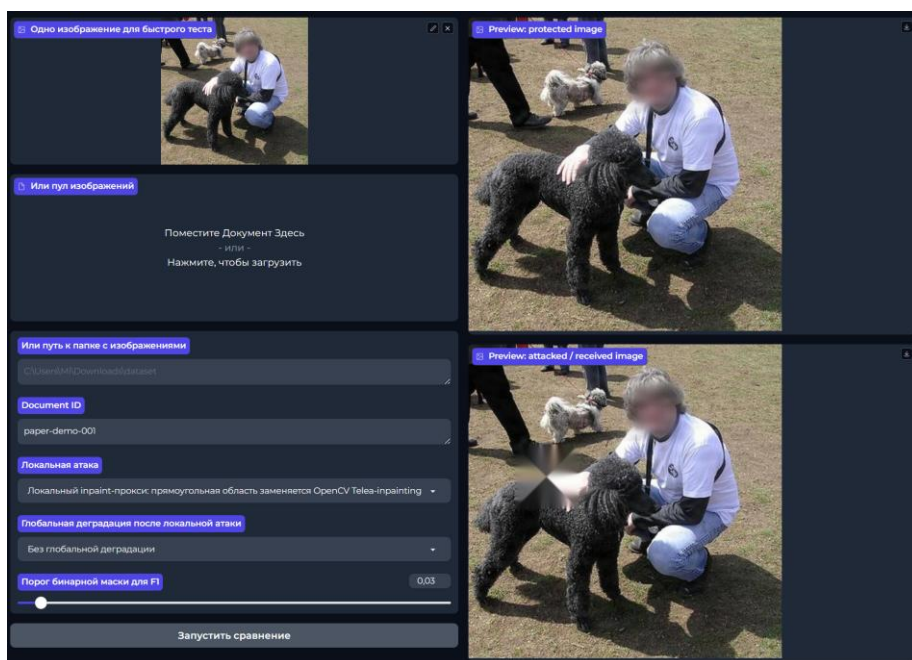


Рисунок 4.7 – Загрузка изображения для дальнейшего сравнения первоначальной и улучшенной версий OmniGuard

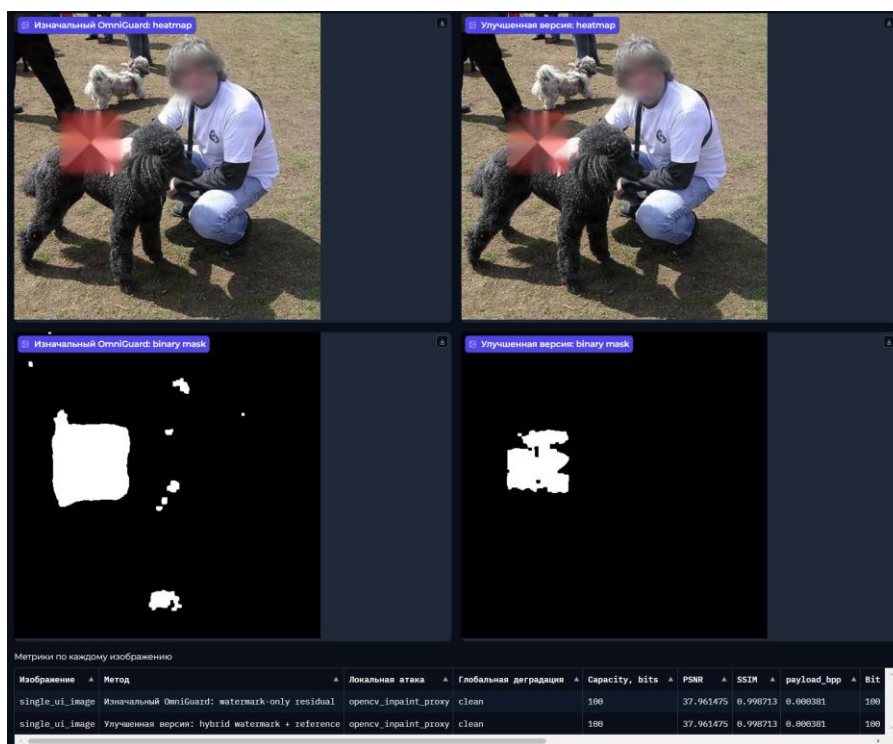


Рисунок 4.8 – Результат сравнения и вывод метрик

4.6 Выводы по разделу

Экспериментальный стенд объединяет демонстрационный интерфейс и инструменты для массовой проверки метода. Он позволяет загрузить изображение, встроить цифровой водяной знак, применить редактирование или деградацию, построить карту вмешательства, извлечь встроенную служебную информацию и сохранить результаты эксперимента.

В отличие от одиночного демонстрационного запуска, стенд поддерживает пакетную обработку и сравнение методов по единому протоколу. Поэтому он является связующим элементом между алгоритмическим описанием усовершенствованного метода и численными экспериментами, приведенными в следующем разделе.

5. Экспериментальная часть

Экспериментальная часть посвящена проверке устойчивости встроенной служебной информации и точности локализации вмешательств. Для сравнения использовались усовершенствованный OmniGuard, метод замены наименее значимых битов LSB и метод на основе дискретного косинусного преобразования. Отдельно проверялось отличие исходного OmniGuard от усовершенствованной версии, поскольку именно эта часть напрямую связана с целью повышения точности детектирования сгенерированных объектов на изображениях.

Основная серия экспериментов выполнена по полному файлу результатов: 1000 изображений, 3 метода и 13 режимов проверки для каждого метода. Всего проанализировано 39000 строк экспериментальных данных. Дополнительно была проведена серия с локальной дорисовкой области, имитирующей современный сценарий содержательного редактирования изображения. Основными метриками стали BER и точность восстановления битов для оценки извлечения служебной метки, а также F1 и AUC для оценки локализации области вмешательства.

5.1 Постановка эксперимента

Для каждого изображения сначала выполнялось встраивание цифрового водяного знака, затем к контейнеру применялся один из режимов проверки: отсутствие атаки, JPEG-сжатие, шумовые воздействия, случайное кадрирование, изменение яркости, медианная фильтрация и уменьшение с последующим восстановлением размера. После этого извлекалась встроенная служебная информация и рассчитывались PSNR, SSIM, bpr и BER. Общие параметры эксперимента приведены в таблице 5.1.

Таблица 5.1 – Общие параметры эксперимента

Параметр	Значение
----------	----------

Количество изображений	1000
Количество строк результатов	39000
Сравниваемые методы	OmniGuard, LSB, ДКП
Режимы проверки	13: без атаки и 12 воздействий
Основные метрики	PSNR, SSIM, bpp, BER
Размер контейнерного изображения	256 × 256

5.2 Незаметность контейнерного изображения

По визуальным метрикам наилучшие значения показывает LSB: метод почти не меняет изображение, так как работает с младшими битами пикселей. ДКП также сохраняет высокие PSNR и SSIM. У OmniGuard PSNR ниже, но SSIM остается высоким, что говорит о сохранении общей структуры изображения. Поэтому качество контейнера нужно рассматривать вместе с устойчивостью извлечения: высокая незаметность сама по себе не гарантирует сохранность встроенных служебных данных после обработки.

BER без внешних воздействий также оказался низким: у LSB он равен 0, у OmniGuard – 0,000050, у ДКП – 0,004794.

Качество контейнерных изображений без атак приведено в таблице 5.2.

Таблица 5.2 – Качество контейнерного изображения без атак

Метод	PSNR	SSIM	bpp	Биты
OmniGuard	39,394	0,998534	0,000381	100
ДКП	57,754	0,999977	0,000519	136
LSB	88,775	1,000000	0,000519	136

5.3 Устойчивость встроенной служебной информации

Главный результат эксперимента проявляется после атак. Средний BER у OmniGuard составил 0,005168, что соответствует точности восстановления 99,483 %. Для ДКП средний BER равен 0,214221, а для LSB – 0,519929. Иначе

говоря, ошибка восстановления у ДКП примерно в 41,45 раза выше, чем у OmniGuard, а у LSB – примерно в 100,60 раза выше.

Медиана BER у OmniGuard равна нулю. Это означает, что более чем в половине проверок после атак встроенная служебная информация восстанавливалась без ошибок. У ДКП медиана составляет 0,198529, а у LSB – 0,507353, что близко к случайному восстановлению битовой строки.

Сводные показатели устойчивости методов после атак приведены в таблице 5.3.

Таблица 5.3 – Устойчивость методов после атак

Метод	Средний BER	Bit Accuracy	Медиана BER	95–й процентиль
OmniGuard	0,005168	99,483 %	0,000000	0,030000
ДКП	0,214221	78,578 %	0,198529	0,463235
LSB	0,519929	48,007 %	0,507353	1,000000

5.4 BER по видам воздействий

OmniGuard показывает лучший результат в 11 из 12 видов атак. Особенно заметное преимущество наблюдается при JPEG–сжатии, гауссовом шуме, кадрировании, медианной фильтрации и масштабировании. Единственное исключение – импульсный шум «соль и перец» (Salt–and–Pepper), где LSB имеет меньший BER. Однако по остальным воздействиям LSB быстро теряет встроенную служебную информацию, поэтому этот частный случай не меняет общего вывода.

Средний BER по отдельным видам воздействий приведен в таблице 5.4.

Таблица 5.4 – Средний BER по видам воздействий

Воздействие	OmniGuard	ДКП	LSB	Лучший метод

JPEG - сжатие, качество 100	0,000070	0,004846	0,488426	OmniGuard
JPEG - сжатие, качество 90	0,000100	0,005794	0,499478	OmniGuard
JPEG - сжатие, качество 50	0,000570	0,461941	0,495654	OmniGuard
Гауссовское зашумление 0.01	0,000070	0,015426	0,510757	OmniGuard
Импульсный шум «соль и перец»	0,019730	0,291066	0,007353	LSB
Случайное кадрирование 10 %	0,013330	0,460904	0,501051	OmniGuard
Увеличение яркости на 15	0,002750	0,044147	0,949404	OmniGuard
Уменьшение яркости на 15	0,003870	0,033735	0,936221	OmniGuard
Умножение яркости на 1,5	0,021160	0,316868	0,491949	OmniGuard
Умножение яркости на 0,8	0,000280	0,039294	0,521426	OmniGuard
Медианная фильтрация 3×3	0,000050	0,440750	0,338412	OmniGuard
Уменьшение и восстановление размера 2×	0,000040	0,455882	0,499015	OmniGuard

5.5 Сравнение исходного и усовершенствованного OmniGuard

Наиболее важная для цели работы серия экспериментов была связана со сравнением исходного OmniGuard и усовершенствованной версии. Здесь оценивалась не только сохранность служебной метки, а прежде всего качество локализации области вмешательства. Для обоих вариантов использовались одни и те же 50 изображений, локальная дорисовка области, отсутствие дополнительной глобальной деградации и единый порог построения маски. Поэтому различия в F1 и AUC связаны с алгоритмом построения карты вмешательства, а не с разными входными условиями.

В исходном режиме карта строилась только по нарушению локального водяного знака. В усовершенствованной версии применялась гибридная карта, объединяющая нарушение водяного знака и сравнение с защищенной опорной версией изображения. Поэтому основные показатели качества контейнера и извлечения служебной битовой метки в большинстве случаев совпадают, а основное различие проявляется в метриках локализации: F1-score, AUC и доле области, отмеченной как измененная.

Результаты разделены на три небольшие таблицы. В первой таблице приведены показатели извлечения служебной битовой метки, во второй — основная метрика локализации F1-score, в третьей — AUC и доля области, которую метод относит к измененной. Такое разделение позволяет не перегружать одну широкую таблицу и одновременно сохранить все сравнительные показатели.

Сводные показатели извлечения служебной битовой метки приведены в таблице 5.5.

Таблица 5.5 – Извлечение служебной битовой метки при различных искажающих воздействиях

Глобальная деградация	Bit Accuracy, %	BER
Без деградации	99,96	0,0004
JPEG Q=85	99,94	0,0006

JPEG Q=70	99,90	0,0010
Гауссовский шум $\sigma=10$	99,94	0,0006
Гауссовский шум $\sigma=25$	99,08	0,0092
Яркость +30 %	99,22	0,0078
Цветовые искажения	99,10	0,0090
Импульсный шум	98,06	0,0194
Случайное выпадение пикселей	93,96	0,0604

Сравнение исходного и усовершенствованного OmniGuard по F1-score приведено в таблице 5.6.

Таблица 5.6 – Сравнение исходного и усовершенствованного OmniGuard по F1-score

Глобальная деградация	F1: исходный / усовершенствованный	Прирост F1
Без деградации	0,649923 / 0,825061	+0,175138
JPEG Q=85	0,078349 / 0,802141	+0,723792
JPEG Q=70	0,069290 / 0,790917	+0,721627
Гауссовский шум $\sigma=10$	0,115911 / 0,738621	+0,622709
Гауссовский шум $\sigma=25$	0,080768 / 0,626265	+0,545496
Яркость +30 %	0,367811 / 0,350889	−0,016922
Цветовые искажения	0,341175 / 0,380236	+0,039061
Импульсный шум	0,251515 / 0,600876	+0,349361
Случайное выпадение пикселей	0,210254 / 0,347588	+0,137334

Сравнение по AUC и доле отмеченной области приведено в таблице 5.7.

Таблица 5.7 – Сравнение исходного и усовершенствованного OmniGuard по AUC и доле отмеченной области

Глобальная деградация	AUC: исходный / усовершенствованный	Доля области: исходный / усовершенствованный
Без деградации	0,999818 / 0,999942	0,088545 / 0,029607
JPEG Q=85	0,517710 / 0,958029	0,225978 / 0,028609
JPEG Q=70	0,501476 / 0,953437	0,226958 / 0,030240
Гауссовский шум $\sigma=10$	0,701208 / 0,939612	0,566492 / 0,029503
Гауссовский шум $\sigma=25$	0,522385 / 0,841649	0,560906 / 0,051544

Яркость +30 %	0,970602 / 0,942167	0,184348 / 0,152394
Цветовые искажения	0,977603 / 0,952110	0,194562 / 0,148150
Импульсный шум	0,963429 / 0,981367	0,279230 / 0,098745
Случайное выпадение пикселей	0,870837 / 0,912874	0,324157 / 0,158297

Из таблицы 5.5 видно, что качество извлечения служебной битовой метки для исходного и усовершенствованного режимов совпадает в каждом сценарии. Это ожидаемо, так как в данном сравнении изменялся именно способ построения карты вмешательства. Поэтому значения Bit Accuracy и BER одинаковы для обоих вариантов в рамках каждой деградации.

Основное отличие проявляется в F1-score. Улучшенный OmniGuard показывает более высокий F1 в 8 из 9 режимов, включая базовый без глобальной деградации. Если учитывать только режимы с дополнительными искажающими воздействиями, улучшенная версия оказывается лучше в 7 из 8. Это означает, что гибридная карта вмешательства точнее согласуется с эталонной областью изменения в большинстве случаев.

Самое большое улучшение наблюдается при JPEG-сжатии. При JPEG Q = 85 F1-score исходного OmniGuard составляет 0,078349, тогда как у улучшенной версии – 0,802141. Прирост равен 0,723792. При JPEG Q = 70 результат аналогичен: F1 увеличивается с 0,069290 до 0,790917. Это означает, что исходный способ построения карты по нарушению локального водяного знака плохо переносит JPEG-сжатие. А гибридная карта остается значительно более устойчивой.

Сильное преимущество также наблюдается при гауссовском шуме. При шуме $\sigma = 10$ F1-score увеличивается с 0,115911 до 0,738621, а при шуме $\sigma = 25$ – с 0,080768 до 0,626265, где параметр σ означает стандартное отклонение добавляемого гауссовского шума. В обоих случаях исходный режим формирует слишком широкую область подозрительности: средняя доля изображения,

отмеченная как измененная, превышает 0,56. Усовершенствованный вариант строит более компактную маску: 0,029503 при $\sigma = 10$ и 0,051544 при $\sigma = 25$. Это важно, потому что чрезмерно широкая маска приводит к большому числу ложных срабатываний и снижает качество локализации.

При импульсном шуме усовершенствованный OmniGuard также показывает более высокий результат. F1-score возрастает с 0,251515 до 0,600876, а AUC - с 0,963429 до 0,981367. При этом доля области, отмеченной как измененная, уменьшается с 0,279230 до 0,098745. Следовательно, улучшенная версия не только лучше отделяет измененные пиксели от неизмененных, но и строит менее избыточную маску.

Для случайного выпадения пикселей улучшение выражено слабее, но сохраняется. F1-score увеличивается с 0,210254 до 0,347588, AUC - с 0,870837 до 0,912874. При этом средняя доля отмеченной области снижается примерно в два раза: с 0,324157 до 0,158297. Это говорит о том, что при более грубом повреждении изображения задача локализации становится сложнее для обоих вариантов, однако усовершенствованная версия все равно сохраняет преимущество.

При повышении яркости на 30 % исходный OmniGuard показывает немного более высокий F1-score: 0,367811 против 0,350889 у усовершенствованной версии. Разница составляет 0,016922, то есть является небольшой на фоне общего преимущества улучшенного метода в остальных сценариях. При цветовых искажениях усовершенствованный вариант имеет более высокий F1-score: 0,380236 против 0,341175, но AUC у него немного ниже. Это означает, что в отдельных цветовых преобразованиях гибридная карта может быть более чувствительной к изменению внешнего вида изображения, однако по основной метрике локализации F1 она все равно показывает прирост.

В среднем по всем рассмотренным режимам F1-score исходного OmniGuard составляет 0,240555, а у усовершенствованной версии - 0,606955. Абсолютный

прирост равен 0,366400. Если учитывать только режимы с глобальными деградациями, средний F1-score увеличивается с 0,189384 до 0,579692, то есть на 0,390307. Это более чем в три раза выше относительно исходного результата.

Среднее значение AUC по режимам с деградациями также возрастает: с 0,753156 у исходного OmniGuard до 0,935156 у усовершенствованной версии. Это показывает, что улучшенная схема лучше ранжирует пиксели по вероятности принадлежности к измененной области. Средняя доля области, отмеченной как измененная, при этом снижается с 0,320329 до 0,087185. Само по себе уменьшение этой доли не является гарантией качества, однако вместе с ростом F1-score оно показывает, что маска становится не просто меньше, а точнее.

Таким образом, эксперимент подтверждает преимущество усовершенствованного OmniGuard именно в той части, которая напрямую связана с целью работы, — в локализации сгенерированных или отредактированных областей изображения. Новый вариант не ухудшает извлечение служебной битовой метки, поскольку значения Bit Accuracy и BER остаются одинаковыми с исходным режимом в одинаковых условиях. При этом гибридная карта вмешательства значительно повышает качество локализации в большинстве проверенных сценариев, особенно при JPEG-сжатии, гауссовском шуме и импульсном шуме. В итоге, по совокупности результатов усовершенствованный OmniGuard показывает более устойчивое и точное поведение.

5.6 Выводы по эксперименту

Проведенный эксперимент показывает разные профили методов. LSB лучше всего сохраняет визуальное качество изображения, но после большинства атак его BER находится около 0,5 или выше, поэтому встроенная служебная информация становится ненадежной. ДКП устойчивее LSB и хорошо переносит часть мягких воздействий, но заметно ухудшается при сильном JPEG-сжатии, кадрировании, фильтрации и масштабировании.

OmniGuard показывает наиболее сильный результат по основной метрике устойчивости. Несмотря на более низкий PSNR, метод сохраняет высокий SSIM и обеспечивает значительно меньший BER после атак. Для задачи проверки сгенерированных объектов это важнее изолированно высокого PSNR, поскольку цифровой водяной знак должен не только быть малозаметным, но и сохранять встроенные данные после типовой обработки изображения.

Дополнительное сравнение исходного и усовершенствованного OmniGuard показывает, что внесенные изменения сильнее всего влияют на качество локализации. При одинаковых значениях PSNR, SSIM, BER и точности восстановления битов улучшенная версия дает более высокую F1-score и формирует менее избыточную маску вмешательства. Это подтверждает, что доработка не ухудшает качество встраивания и извлечения служебной информации, но делает результат локализации более точным и пригодным для дальнейшей оценки.

Выводы

В ходе выполнения работы были рассмотрены алгоритмы встраивания цифровых водяных знаков, применимые для распознавания и проверки сгенерированных объектов на изображениях. В литературном обзоре были изучены классические пространственные и частотные методы, а также современные нейросетевые подходы, ориентированные на совмещение защиты авторства и локализации вмешательств. По результатам обзора в качестве базового метода был выбран OmniGuard, поскольку он поддерживает гибридную схему цифрового водяного маркирования и лучше соответствует задаче поиска измененных областей на изображении.

Практическая часть была связана с адаптацией и дальнейшим развитием исходной реализации OmniGuard. Сначала исследовательский прототип был приведен к воспроизводимому локальному запуску: исправлены зависимости, загрузка моделей, работа с вычислительным устройством, служебные пути и логика интерфейса. После этого была выполнена алгоритмическая модификация метода. В усовершенствованную версию добавлены служебные данные, привязка к идентификатору изображения, HMAC-проверка целостности, коррекция битовых ошибок с помощью кода Хэмминга, гибридная карта вмешательства и пакетный экспериментальный контур.

Для проверки работоспособности был разработан экспериментальный стенд. Он позволяет загружать изображения, встраивать цифровой водяной знак, моделировать редактирование, запускать пакетные эксперименты и сохранять результаты. Это позволило перейти от одиночной демонстрации к воспроизводимой оценке метода на датасетах с изображениями.

Экспериментальное сравнение усовершенствованного метода OmniGuard с методом LSB и методом на основе ДКП показало, что классические методы могут обеспечивать высокую визуальную незаметность, но будут заметно уступать по устойчивости встроенной служебной информации после типовых воздействий.

Как итог, OmniGuard показал наилучший результат по сохранности встроенных данных после атак: средний BER оказался значительно ниже, чем у LSB и ДКП. Это подтверждает, что для задачи проверки изображений после обработки важна не только незаметность встраивания, но и устойчивость извлечения.

Дополнительно было проведено сравнение исходного OmniGuard и усовершенствованной версии. Результаты показали, что обновленная версия сохраняет качество встраивания и извлечения служебной информации, но улучшает локализацию области вмешательства. Среднее значение F1-score выросло с 0,649923 до 0,825061, а улучшенная версия показала более высокий F1-score на 44 из 50 изображений. Это подтверждает, что добавление гибридной карты вмешательства повышает точность выделения измененной области.

Таким образом, в работе была улучшена полухрупкая схема встраивания цифровых водяных знаков, подготовлена ее программная реализация и проведены вычислительные эксперименты для оценки эффективности обновленного варианта метода. Полученные результаты показывают, что предложенная модификация OmniGuard сохраняет устойчивость встроенной служебной информации при типовых искажающих воздействиях и повышает качество локализации фальсификаций по сравнению с исходной версией метода.

Список использованной литературы

1. Lin C. Y., Chang S. F. Semi-fragile watermarking for authenticating JPEG visual content //Security and watermarking of multimedia contents II. – SPIE, 2000. – T. 3971. – C. 140–151.
2. Begum M., Uddin M. S. Digital image watermarking techniques: a review //Information. – 2020. – T. 11. – №. 2. – C. 110.
3. Rakhmawati L., Wirawan W., Suwadi S. A recent survey of self-embedding fragile watermarking scheme for image authentication with recovery capability //EURASIP Journal on Image and Video Processing. – 2019. – T. 2019. – №. 1. – C. 61.
4. Ho C. K., Li C. T. Semi-fragile watermarking scheme for authentication of JPEG images //International Conference on Information Technology: Coding and Computing, 2004. Proceedings. ITCC 2004. – IEEE, 2004. – T. 1. – C. 7–11.
5. Yu X., Wang C., Zhou X. Review on semi-fragile watermarking algorithms for content authentication of digital images //Future Internet. – 2017. – T. 9. – №. 4. – C. 56.
6. Shieh J. M., Lou D. C., Chang M. C. A semi-blind digital watermarking scheme based on singular value decomposition //Computer Standards & Interfaces. – 2006. – T. 28. – №. 4. – C. 428–440.
7. Abdel-Aziz M. M. et al. Blind watermarking of color medical images using hadamard transform and fractional-order moments //Sensors. – 2021. – T. 21. – №. 23. – C. 7845.
8. Ramamurthy N., Varadarajan S. The Robust digital image watermarking using quantization and fuzzy logic approach in DWT domain //arXiv preprint arXiv:1302.4233. – 2013.

9. Bamatraf A., Ibrahim R., Salleh M. N. B. M. Digital watermarking algorithm using LSB //2010 International Conference on Computer Applications and Industrial Electronics. – IEEE, 2010. – C. 155–159.
10. Singh R. K., Shaw D. K., Alam M. J. Experimental studies of LSB watermarking with different noise //Procedia Computer Science. – 2015. – T. 54. – C. 612–620.
11. Lin S. D., Shie S. C., Guo J. Y. Improving the robustness of DCT–based image watermarking against JPEG compression //Computer Standards & Interfaces. – 2010. – T. 32. – №. 1–2. – C. 54–60.
12. Yu L. et al. PM1 steganography in JPEG images using genetic algorithm //Soft Computing. – 2009. – T. 13. – №. 4. – C. 393-400.
13. Mandal J. K., Das D. A novel invisible watermarking based on cascaded PVD integrated LSB technique //International Conference on Eco–friendly Computing and Communication Systems. – Berlin, Heidelberg : Springer Berlin Heidelberg, 2012. – C. 262–268.
14. Paul G., Saha S. K., Burman D. A PVD based high capacity steganography algorithm with embedding in non-sequential position //Multimedia Tools and Applications. – 2020. – T. 79. – №. 19. – C. 13449-13479.
15. Prins J. P., Erkin Z., Lagendijk R. L. Anonymous fingerprinting with robust QIM watermarking techniques //EURASIP Journal on Information Security. – 2007. – T. 2007. – №. 1. – C. 031340.
16. Yassin N. I., Salem N. M., El Adawy M. I. QIM blind video watermarking scheme based on Wavelet transform and principal component analysis //Alexandria Engineering Journal. – 2014. – T. 53. – №. 4. – C. 833–842.
17. Cox I. J. et al. Secure spread spectrum watermarking for multimedia //IEEE transactions on image processing. – 1997. – T. 6. – №. 12. – C. 1673–1687.
18. Pérez–Freire L., Pérez–González F. Spread–spectrum watermarking security //IEEE Transactions on Information Forensics and Security. – 2009. – T. 4. – №. 1. – C. 2–24.

19. Jiansheng M., Sukang L., Xiaomei T. A digital watermarking algorithm based on DCT and DWT //Proceedings. the 2009 international symposium on web information systems and applications (WISA 2009). – Academy publisher, 2009. – C. 104.
20. Al-Haj A. Combined DWT–DCT digital image watermarking //Journal of computer science. – 2007. – T. 3. – №. 9. – C. 740–746.
21. Zhou X., Zhang H., Wang C. A robust image watermarking technique based on DWT, APDCBT, and SVD //Symmetry. – 2018. – T. 10. – №. 3. – C. 77.
22. Abdelhakim A. M. et al. Optimized SVD–based robust watermarking in the fractional Fourier domain //Multimedia Tools and Applications. – 2018. – T. 77. – №. 21. – C. 27895–27917.
23. Urvoy M., Goudia D., Autrusseau F. Perceptual DFT watermarking with improved detection and robustness to geometrical distortions //IEEE Transactions on Information Forensics and Security. – 2014. – T. 9. – №. 7. – C. 1108–1119.
24. Fazli S., Moeini M. A robust image watermarking method based on DWT, DCT, and SVD using a new technique for correction of main geometric attacks //Optik. – 2016. – T. 127. – №. 2. – C. 964–972.
25. Zhang X. et al. Editguard: Versatile image watermarking for tamper localization and copyright protection //Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. – 2024. – C. 11964–11974.
26. Zhang X. et al. Omniguard: Hybrid manipulation localization via augmented versatile deep image watermarking //Proceedings of the Computer Vision and Pattern Recognition Conference. – 2025. – C. 3008–3018.
27. Neekhara P. et al. FaceSigns: Semi-fragile watermarks for media authentication //ACM Transactions on Multimedia Computing, Communications and Applications. – 2024. – T. 20. – №. 11. – C. 1-21.

28. DeepFakes [Электронный ресурс] // GitHub. – URL: <https://github.com/deepfakes/faceswap> (дата обращения: 29.10.2025)
29. Faceswap [Электронный ресурс] // GitHub. – URL: <https://github.com/marekkowalski/faceswap/> (дата обращения: 29.10.2025)
30. Rossler A. et al. Faceforensics++: Learning to detect manipulated facial images //Proceedings of the IEEE/CVF international conference on computer vision. – 2019. – С. 1-11.
31. Hussain S. et al. Adversarial deepfakes: Evaluating vulnerability of deepfake detectors to adversarial examples //Proceedings of the IEEE/CVF winter conference on applications of computer vision. – 2021. – С. 3348-3357.

Приложение А. Описание программной реализации и ключевые фрагменты кода

В приложении приведен основной программный файл экспериментального стенда. В качестве такого файла выбран `omniguard/service.py`, поскольку в нем расположен класс `OmniGuardEngine`, объединяющий ключевые этапы работы системы: защиту изображения, редактирование, извлечение служебной битовой метки, построение карты вмешательства, формирование бинарной маски и расчет сопоставительных метрик. Остальные модули проекта вызываются из этого файла как вспомогательные компоненты: `payload.py` отвечает за служебную битовую метку, `legasy_models.py` – за взаимодействие с моделями `OmniGuard`, `image_ops.py` – за операции с изображениями, `metrics.py` – за расчет метрик.

Полная версия проекта размещена в публичном репозитории GitHub: https://github.com/DumbyDum/VKR_Gadzhiev.

```
from __future__ import annotations
```

```
from datetime import datetime, timezone
```

```
import json
```

```
from pathlib import Path
```

```
from typing import Any
```

```
import cv2
```

```
import numpy as np
```

```
from .editing import EditingResult, build_editor
```

```
from .image_ops import (
```

```
    ensure_mask_uint8,
```

```
    load_image,
```

```
    limit_image_side,
```



```

        normalize_mask,
        overlay_mask,
        resize_image,
        save_image,
        save_mask,
    )

    from .legacy_models import LegacyModelBundle
    from .metrics import changed_pixel_ratio, mae, mse, psnr, rmse, ssim
    from .payload import build_payload_bits, decode_payload_bits
    from .schemas import AnalysisResult, ProtectionResult
    from .settings import RuntimeSettings

    class OmniGuardEngine:

        def __init__(self, settings: RuntimeSettings | None = None):
            self.settings = settings or RuntimeSettings()
            self.settings.ensure_runtime_dirs()
            self.models = LegacyModelBundle(self.settings)
            self._editors: dict[tuple[str, str, bool], object] = {}

        def get_editor(
            self,
            editor_name: str = "auto",
            *,
            model_id: str | None = None,
            allow_download: bool | None = None,
        ):
            resolved_model_id = (model_id or self.settings.inpaint_model_id).strip()
            resolved_allow_download = (

```

```

        self.settings.allow_inpaint_model_download if allow_download is None
    else allow_download
    )
    cache_key = (editor_name, resolved_model_id,
bool(resolved_allow_download))
    if cache_key not in self._editors:
        self._editors[cache_key] = build_editor(
            self.settings,
            editor_name=editor_name,
            model_id=resolved_model_id,
            allow_download=resolved_allow_download,
        )
    return self._editors[cache_key]

def protect_image(
    self,
    image: str | Path | np.ndarray,
    document_id: str,
    output_path: str | Path | None = None,
) -> ProtectionResult:
    source = limit_image_side(load_image(image),
self.settings.max_input_side)
    payload = build_payload_bits(document_id, self.settings.hmac_secret)
    tamper_protected = self.models.embed_tamper_watermark(source)
    final_protected = self.models.embed_payload_bits(
        tamper_protected,
        payload.encoded_bits,
        strength=self.settings.payload_strength,

```

```

    )
    protected_path = None
    if output_path is not None:
        protected_path = save_image(final_protected, output_path)
    metadata = {
        "document_id": document_id,
        "created_at_utc": datetime.now(timezone.utc).isoformat(),
        "payload_bit_length": self.settings.payload_bit_length,
        "payload_strength": self.settings.payload_strength,
        "tamper_mask_scales": list(self.settings.tamper_mask_scales),
    }
    return ProtectionResult(
        protected_image_path=protected_path,
        protected_image=final_protected,
        payload=payload,
        metadata=metadata,
    )

def edit_image(
    self,
    image: str | Path | np.ndarray,
    mask: np.ndarray,
    prompt: str = "",
    *,
    editor_name: str = "auto",
    editor_model_id: str | None = None,
    allow_download: bool | None = None,
) -> EditingResult:

```

```

        source = limit_image_side(load_image(image),
self.settings.max_input_side)
        safe_mask = ensure_mask_uint8(mask)
        editor = self.get_editor(
            editor_name=editor_name,
            model_id=editor_model_id,
            allow_download=allow_download,
        )
        return editor.edit(source, safe_mask, prompt)

def _align_reference_image(
    self,
    reference_image: str | Path | np.ndarray | None,
    target_shape: tuple[int, int],
) -> np.ndarray | None:
    if reference_image is None:
        return None
    reference = limit_image_side(load_image(reference_image),
self.settings.max_input_side)
    target_height, target_width = target_shape
    if reference.shape[:2] != (target_height, target_width):
        reference = resize_image(reference, (target_width, target_height))
    return reference

def _normalize_score_map(
    self,
    score_map: np.ndarray,
    *,

```

```

        low_percentile: float,
        high_percentile: float,
        blur_sigma: float = 0.0,
    ) -> np.ndarray:
        score = np.nan_to_num(score_map.astype(np.float32), nan=0.0,
                               posinf=0.0, neginf=0.0)
        score = np.maximum(score, 0.0)
        if blur_sigma > 0:
            score = cv2.GaussianBlur(score, (0, 0), blur_sigma)
        low = float(np.percentile(score, low_percentile))
        high = float(np.percentile(score, high_percentile))
        if high <= low + 1.0e-6:
            low = float(score.min())
            high = float(score.max())
        if high <= low + 1.0e-6:
            return np.zeros_like(score, dtype=np.float32)
        normalized = (score - low) / (high - low)
        return np.clip(normalized, 0.0, 1.0).astype(np.float32)

    def _reference_difference_map(self, source: np.ndarray, reference:
np.ndarray) -> np.ndarray:
        source_f = source.astype(np.float32) / 255.0
        reference_f = reference.astype(np.float32) / 255.0
        channel_diff = np.abs(source_f - reference_f)
        gray = cv2.cvtColor((channel_diff * 255.0).astype(np.uint8),
cv2.COLOR_RGB2GRAY).astype(np.float32) / 255.0
        max_channel = channel_diff.max(axis=2)
        blended = np.maximum(gray, max_channel)

```

```

return self._normalize_score_map(
    blended,
    low_percentile=65.0,
    high_percentile=99.5,
    blur_sigma=1.2,
)

def _combined_heatmap(
    self,
    source: np.ndarray,
    *,
    reference_image: str | Path | np.ndarray | None,
    analysis_mode: str,
) -> tuple[np.ndarray, np.ndarray, np.ndarray | None, np.ndarray | None]:
    watermark_raw = self.models.reveal_tamper_mask(source,
self.settings.tamper_mask_scales)
    watermark_map = self._normalize_score_map(
        watermark_raw,
        low_percentile=80.0,
        high_percentile=99.8,
        blur_sigma=1.0,
    )
    reference = self._align_reference_image(reference_image,
source.shape[:2])
    reference_map = self._reference_difference_map(source, reference) if
reference is not None else None

    if analysis_mode == "reference" and reference_map is not None:

```

```

        combined = reference_map
    elif analysis_mode == "hybrid" and reference_map is not None:
        combined = np.clip(0.35 * watermark_map + 0.65 * reference_map,
0.0, 1.0)

        combined = np.maximum(combined, reference_map * 0.95)
    else:
        combined = watermark_map

    combined = cv2.GaussianBlur(combined.astype(np.float32), (0, 0), 1.1)
    combined = np.clip(combined, 0.0, 1.0)
    return combined, watermark_map, reference_map, reference

def _build_binary_mask(
    self,
    heatmap: np.ndarray,
    *,
    reference_map: np.ndarray | None,
    threshold: float,
) -> np.ndarray:
    if reference_map is None:
        binary = (heatmap >= threshold).astype(np.uint8) * 255
    else:
        reference_u8 = np.clip(reference_map * 255.0, 0.0,
255.0).astype(np.uint8)
        otsu_value, _ = cv2.threshold(reference_u8, 0, 255,
cv2.THRESH_BINARY + cv2.THRESH_OTSU)
        reference_threshold = max(threshold, float(otsu_value) / 255.0)
        direct_mask = reference_map >= max(0.08, reference_threshold * 0.7)

```

```

        combined_mask = heatmap >= max(threshold, reference_threshold *
0.85)

        binary = np.logical_and(direct_mask, np.logical_or(combined_mask,
reference_map >= 0.22)).astype(np.uint8) * 255

    if binary.any():
        binary = cv2.morphologyEx(binary, cv2.MORPH_OPEN, np.ones((3,
3), dtype=np.uint8))
        binary = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, np.ones((7,
7), dtype=np.uint8))
        binary = cv2.dilate(binary, np.ones((3, 3), dtype=np.uint8), iterations=1)
    return binary

def _comparison_metrics(self, source: np.ndarray, reference: np.ndarray |
None) -> dict[str, Any]:
    if reference is None:
        return {}
    return {
        "mse_vs_reference": mse(reference, source),
        "mae_vs_reference": mae(reference, source),
        "rmse_vs_reference": rmse(reference, source),
        "psnr_vs_reference": psnr(reference, source),
        "ssim_vs_reference": ssim(reference, source),
        "changed_pixel_ratio_vs_reference": changed_pixel_ratio(reference,
source),
    }

def analyze_image(

```



```

self,
image: str | Path | np.ndarray,
expected_document_id: str | None = None,
reference_bits: list[int] | None = None,
output_dir: str | Path | None = None,
reference_image: str | Path | np.ndarray | None = None,
analysis_mode: str = "hybrid",
threshold_override: float | None = None,
) -> AnalysisResult:
    source = limit_image_side(load_image(image),
self.settings.max_input_side)
    predicted_bits = self.models.decode_payload_bits(source)
    payload_result = decode_payload_bits(
        predicted_bits,
        self.settings.hmac_secret,
        expected_document_id=expected_document_id,
        reference_bits=reference_bits,
    )
    threshold = threshold_override if threshold_override is not None else
self.settings.tamper_mask_threshold
    heatmap_float, watermark_map, reference_map, aligned_reference =
self._combined_heatmap(
        source,
        reference_image=reference_image,
        analysis_mode=analysis_mode,
    )
    binary_mask = self._build_binary_mask(
        heatmap_float,

```

```

        reference_map=reference_map,
        threshold=threshold,
    )
    heatmap_uint8 = np.clip(heatmap_float * 255.0, 0.0,
255.0).astype(np.uint8)
    tamper_ratio = float(normalize_mask(binary_mask).mean())
    tamper_heatmap_path = None
    binary_mask_path = None
    if output_dir is not None:
        output_dir = Path(output_dir)
        output_dir.mkdir(parents=True, exist_ok=True)
        tamper_heatmap_path = save_image(overlay_mask(source,
heatmap_uint8), output_dir / "tamper_heatmap.png")
        binary_mask_path = save_mask(binary_mask, output_dir /
"tamper_mask.png")
        comparison_metrics = self._comparison_metrics(source,
aligned_reference)
        metadata = {
            "expected_document_id": expected_document_id,
            "threshold": threshold,
            "scales": list(self.settings.tamper_mask_scales),
            "analysis_mode": analysis_mode,
            "reference_image_used": aligned_reference is not None,
            "watermark_score_mean": float(watermark_map.mean()),
            "watermark_score_max": float(watermark_map.max()),
            "reference_score_mean": float(reference_map.mean()) if reference_map
is not None else None,

```

```
        "reference_score_max": float(reference_map.max()) if reference_map is
not None else None,
```

```
    }
```

```
    return AnalysisResult(
        payload=payload_result,
        tamper_heatmap_path=tamper_heatmap_path,
        binary_mask_path=binary_mask_path,
        tamper_heatmap=heatmap_uint8,
        binary_mask=binary_mask,
        tamper_score_mean=float(heatmap_float.mean()),
        tamper_score_max=float(heatmap_float.max()),
        tamper_ratio=tamper_ratio,
        metadata=metadata,
        comparison_metrics=comparison_metrics,
    )
```

```
def save_json(self, data: dict[str, Any], path: str | Path) -> Path:
```

```
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as handle:
        json.dump(data, handle, ensure_ascii=False, indent=2)
    return path
```

```
def save_protection_bundle(
```

```
    self,
    result: ProtectionResult,
    image_path: str | Path,
    metadata_path: str | Path,
```

```

) -> tuple[Path, Path]:
    saved_image = save_image(result.protected_image, image_path)
    saved_meta = self.save_json(result.to_dict(), metadata_path)
    result.protected_image_path = saved_image
    return saved_image, saved_meta

def save_analysis_bundle(
    self,
    result: AnalysisResult,
    output_dir: str | Path,
) -> tuple[Path, Path, Path]:
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    heatmap_path = save_mask(result.tamper_heatmap, output_dir /
"analysis_heatmap.png")
    mask_path = save_mask(result.binary_mask, output_dir /
"analysis_mask.png")
    report_path = self.save_json(result.to_dict(), output_dir /
"analysis_report.json")
    return heatmap_path, mask_path, report_path

def protection_summary(self, result: ProtectionResult) -> dict[str, Any]:
    summary = result.to_dict()
    summary["payload"]["record"]["issued_at_utc"] =
result.payload.record.issued_at_utc.isoformat()
    return summary

def analysis_summary(self, result: AnalysisResult) -> dict[str, Any]:

```

```
summary = result.to_dict()
if result.payload.record is not None:
    summary["payload"]["record"]["issued_at_utc"] =
result.payload.record.issued_at_utc.isoformat()
return summary
```